Efficient Batch Threshold Encryption Using Partial Fraction Techniques

Dan Boneh[1], Rohit Nema[1], Arnab Roy[2], and Ertem Nusret Tas[3]

[1] Stanford University {dabo,rnema}@cs.stanford.edu
[2] Mysten Labs arnab@mystenlabs.com
[3] a16z Crypto Research ntas@a16z.com

Abstract. Batch encryption enables a holder of the secret key to publish a succinct pre-decryption key for a set of ciphertexts, such that exactly that set can be decrypted while other ciphertexts remain secret. Existing constructions either rely on epochs or, when epochless, suffer from large public parameters, and are vulnerable to censorship. In this work, we present an epochless, censorship-resistant batch encryption scheme with linear-sized public parameters, constant-sized pre-decryption keys and ciphertexts, and efficient batch decryption. Our construction extends the partial fraction techniques of Jutla, Nema, and Roy's threshold encryption scheme: we exploit partial fraction decomposition such that publishing a single group element as the pre-decryption key lets the decryptor decrypt all ciphertexts in the batch. We prove CCA security of our scheme, and show how to thresholdize it. Our results directly benefit applications such as encrypted mempools for MEV mitigation and time-lock encrypted storage.

Table of Contents

1 Introduction . . . 3
- 1.1 Applications . . . 5

2 Technical Overview . . . 6
- 2.1 Overview of the Threshold Encryption Construction in [24] . . . 7
- 2.2 Starting Point: Index-Dependent Batch Encryption . . . 8
- 2.3 Removing Index Dependence . . . 9
- 2.4 Amortizing Pairings and Group Operations . . . 11
- 2.5 CCA-2 Security . . . 12
- 2.6 Thresholdizing Batch Encryption . . . 12
- 2.7 Shrinking Ciphertext Size . . . 13
- 2.8 Idealized Model Tradeoffs . . . 14

3 Preliminaries . . . 14
- 3.1 Notation . . . 14
- 3.2 Partial Fraction Decomposition . . . 14
- 3.3 Batch Encryption . . . 15
- 3.4 Batch Threshold Encryption . . . 15
- 3.5 Non-Interactive Zero Knowledge Proofs (NIZKs) . . . 17

4 Construction . . . 19
- 4.1 Batch Encryption . . . 20
- 4.2 Batch Threshold Encryption . . . 25
- 4.3 NIZK for Ciphertext Relation . . . 28

5 Batch Encryption With Shorter Ciphertexts . . . 32
- 5.1 Correctness . . . 34
- 5.2 Decryption Complexity . . . 35
- 5.3 Security . . . 35

6 Experimental Evaluation . . . 36

7 Acknowledgments . . . 36

A Additional Proofs . . . 39
- A.1 Hardness of q-StatRankDef (Assumption 1) . . . 39
- A.2 Generic Hardness of q-QuadRankDef (Assumption 2) . . . 40
- A.3 CCA-2 Security of $\mathsf{BE}_{\mathrm{short}}$ . . . 40

1 Introduction

A *batch encryption* scheme allows a party with the secret key to release a single, short *pre-decryption key* for a chosen set of ciphertexts; anyone with this key can decrypt exactly that set, while ciphertexts outside the set remain secret (even if the adversary has access to a pre-decryption oracle). Thus, instead of publishing one decryption key per ciphertext, the key holder publishes one compact key per batch, which is the main efficiency gain. Formally, a batch encryption scheme is defined by four PPT algorithms $(\mathsf{Setup}, \mathsf{Enc}, \mathsf{PreDec}, \mathsf{Dec})$:

- $\mathsf{Setup}(1^\lambda, 1^\ell) \to (\mathsf{ek}, \mathsf{sk}, \mathsf{dk})$: given the security parameter λ and a maximum batch size ℓ, outputs a public encryption key ek, secret key sk, and a public decryption key dk;
- $\mathsf{Enc}(\mathsf{ek}, m) \to \mathsf{ct}$: encrypts m with respect to ek;
- $\mathsf{PreDec}(\mathsf{sk}, (\mathsf{ct}_i)_{i\in B}) \to \mathsf{sbk}$: given a secret key sk and a set of up to ℓ ciphertexts, outputs the corresponding *short* pre-decryption key sbk;
- $\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in B}) \to (m_i)_{i\in B}$: decrypts a set of ciphertexts given the corresponding pre-decryption key sbk and public decryption key dk.

The key property of batch encryption is the succinctness of the pre-decryption key – its size must be at most poly-logarithmic in the maximum batch size ℓ. Otherwise, as a trivial scheme, the pre-decryption algorithm could simply decrypt the ciphertexts using sk and publish $|B|$ messages in place of the pre-decryption key, resulting in $O(\ell)$ communication.

Batch identity-based encryption with epochs. There have been two approaches to construct batch encryption, which was first introduced as a primitive by Choudhuri et al. [16]. Their construction is in fact a *batch identity-based encryption* scheme, where messages are encrypted with respect to an *identity tag* r and an *epoch id* e. The PreDec algorithm takes a secret key, an epoch id e, and a set of identity tags $\{r_i\}_{i\in B}$, and publishes a *short* pre-decryption key associated with epoch e and the given set of identities (as opposed to one key per tag). This key allows the decryption of every message encrypted to an identity tag in $\{r_i\}_{i\in B}$ and the epoch id e, but no other ciphertext. While the original scheme [16] ran a fresh setup before every epoch, later work [30,5,17] reduced this to a one-time setup, while still permitting one key per epoch.

A major limitation of the batch IBE schemes [30,5,17] is that at most a *single* pre-decryption key can be published per epoch without compromising security. To understand why this is a problem, consider a ciphertext ct encrypted to an identity tag r and an epoch id e. If a pre-decryption key is published for the epoch id e and a batch excluding r, then ct cannot ever be decrypted without compromising security; as no other pre-decryption key can be published for epoch e. Hence, to be included in a future batch, the message underlying ct must be re-encrypted under a different epoch id. This in turn degrades the usability of systems based on batch encryption (*e.g.*, encrypted mempools, see Section 1.1.1).

Index-dependent batch encryption. To remove epochs, Bormet et al. present an alternative batch encryption construction based on key-homomorphic PRFs that are made puncturable [14]. In their scheme, called BEAT-MEV, messages are encrypted with respect to a public key pk and an *index* i chosen by the user at the encryption time. The PreDec algorithm takes a secret key and a set of ciphertexts $(\mathsf{ct}_i)_{i\in B}$ with *distinct* indices, and publishes a short pre-decryption key that allows the decryption of every ciphertext in this set, but nothing else (CCA-2 security). The indices in BEAT-MEV are different from the identity tags used by the (batch) IBE schemes: a pre-decryption

key for a batch containing an index-i ciphertext decrypts only that ciphertext, not every ciphertext with index i. Therefore, ciphertexts in different batches can share the same indices.

As BEAT-MEV requires each ciphertext in a batch to have *distinct* indices; we say that BEAT-MEV is *index-dependent*. Moreover, the size of the public parameters scales linearly [19,13] in the size of the index space. Thus, towards smaller public parameters, BEAT-MEV constrains the index space to the set $\{1, \ldots, \ell\}$. This, however, increases the probability of index collision among the ciphertexts submitted to a batch of size ℓ, which in turn prevents the generation of the pre-decryption key. To mitigate this problem, BEAT-MEV groups transactions into multiple sub-batches, each consisting of ciphertexts with distinct indices, and publishes one pre-decryption key per sub-batch, *i.e.*, $O(\log \ell)$ keys in expectation when the indices are randomly chosen [14, §5.3]. Unfortunately, sub-batching cannot prevent an adversarial encryptor from submitting $\Omega(\log \ell)$ ciphertexts to a batch, all sharing the same index as a target ciphertext ct, with the aim of excluding ct from the batch. Thus, the index-dependency of BEAT-MEV results in a so-called *censorship* vulnerability (called succinctness vulnerability in [10, §1.3]). In contrast, the batch IBE schemes are not susceptible to the same attack: when the adversary cannot pick the same identity tag as a target ciphertext (which can be enforced, see [17,16,5,30,14]), its only option is to flood the batch with $O(\ell)$ ciphertexts, exponentially more than $O(\log \ell)$.

In summary, batch IBE schemes are limited by epochs, whereas BEAT-MEV and the constructions building on it [13,3] are index-dependent.

Construction	$\lvert\mathsf{crs}\rvert$	$\lvert\mathsf{sbk}\rvert$	$\lvert\mathsf{ct}\rvert$	**Epochless**	**II**	**Model**
Batch IBE [17,5,30]	$2\lvert\mathbb{G}_1\rvert + \ell\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_2\rvert$	$3\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	No	Yes	GGM
TrX [20]	$\kappa\ell\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_2\rvert$	$2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	Yes	GGM
Gong et al. [22, Cor. 4.5]	$5\lvert\mathbb{G}_1\rvert + \ell\lvert\mathbb{G}_2\rvert + \lvert\mathbb{G}_T\rvert$	$2\lvert\mathbb{G}_2\rvert + \lvert\mathbb{Z}_p\rvert$	$2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	No	Yes	Plain
Gong et al. [22, Cor. D.6]	$4\lvert\mathbb{G}_1\rvert + \ell\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_2\rvert$	$3\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	No	Yes	GGM
BEAT-MEV [14]	$(\ell+1)\lvert\mathbb{G}_1\rvert + \ell^2\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_1\rvert$	$3\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	No	AGM+ROM
BEAST-MEV [13]	$\Theta(\ell)\lvert\mathbb{G}_1\rvert + \Theta(\ell)\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_1\rvert$	$3\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	No	AGM+ROM
Agarwal et al. [3]	$\Theta(\ell)\lvert\mathbb{G}_1\rvert + \Theta(\ell)\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_2\rvert$	$2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	No	GGM
BTX [4] Simple BTE [25]	$\lvert\mathbb{G}_T\rvert + (2\ell-1)\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_1\rvert$	$\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	Yes	AGM+ROM
Section 4	$2\lvert\mathbb{G}_1\rvert + \ell\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_1\rvert$	$2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	Yes	AGM+ROM
Section 5	$\lvert\mathbb{G}_1\rvert + (2\ell+1)\lvert\mathbb{G}_2\rvert$	$\lvert\mathbb{G}_1\rvert$	$\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_T\rvert$	Yes	Yes	AGM+ROM

Table 1: Comparison of batch encryption systems based on elliptic curves. $\lvert\mathsf{crs}\rvert$ denotes the total size of public parameters (which in our case comprises ek and dk), $\lvert\mathsf{sbk}\rvert$ denotes the pre-decryption key size, and $\lvert\mathsf{ct}\rvert$ denotes the ciphertext size. ℓ denotes the maximum batch size, and κ denotes the total number of batches following the initial setup. We denote the sizes of the elliptic-curve group elements by $\lvert\mathbb{G}_i\rvert$, $i = 1, 2, T$. II signifies that the scheme is index-independent and not susceptible to the censorship vulnerability described above. Silent setup functionality is not shown on the table, and is achieved by [13]. NIZK proofs in the case of BEAT-MEV [14] are of size $(3\lvert\mathbb{G}_1\rvert + 2\lvert\mathbb{F}_p\rvert)$, whereas in our case, they are $(2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{F}_p\rvert)$ and $(\lvert\mathbb{G}_1\rvert + \lvert\mathbb{F}_p\rvert)$ for the constructions in Sections 4 and 5 respectively (excluded from the table). However, our batch threshold encryption has secret key size of $\ell\lvert\mathbb{F}_p\rvert$. For BTX and Simple BTE, that are follow-up works to ours, we report the non-threshold public parameters to match the table convention.

Our Contributions. In this work, we present an index-independent, epochless, and concretely efficient batch (threshold) encryption scheme with $O(\ell)$-sized public parameters, and $O(1)$-sized encryption key, pre-decryption key and ciphertexts. For comparison of our batch encryption construction to prior work, see Table 1. In particular, we obtain the shortest CRS (*i.e.*, the encryption

and decryption key collectively), ciphertext and pre-decryption key sizes, even when compared to epoch-based schemes.

Our constructions are enabled by a novel extension of the "partial fraction" technique used by Jutla, Nema and Roy to construct an efficient threshold encryption scheme [24]. Section 2 provides a detailed overview of their technique and its application to our work. We prove security under a falsifiable assumption from [24] in the algebraic group model and the random oracle model.

We formally define batch (threshold) encryption in Section 3. We then present our batch encryption and batch threshold encryption constructions and prove their security in Sections 4.1 and 4.2 respectively. We also prove an important property of *robustness* for the threshold construction. In Section 5, we present a variant of the scheme in Section 4.1 with shorter ciphertexts at the expense of a longer CRS. It can be thresholdized in the same manner as Section 4.2.

In Section 6, we report on an implementation of the constructions in Sections 4.1 and 5. Despite the codebase being unoptimized, our benchmarks show that the scheme is fast and extremely practical for real-world use.

Additional Related Work. TrX [20] gives the first implementation of encrypted mempools (see Section 1.1) based on batch IBE (specifically, a variant of the construction in [17]) in the context of a BFT consensus protocol. It improves upon the prior batch IBE protocols by reducing the ciphertext size. It also resolves the limitation of epochs, albeit at the expense of larger public parameters that scale linearly in the number of batches (denoted by κ on Table 1). Boneh, Laufer, and Tas [10] present the first epochless batch IBE scheme with succinct public keys from the Learning with Errors (LWE) assumption. Unlike the prior schemes based on elliptic curves, their construction is lattice-based with much larger parameters in practice. BEAST-MEV [13] improves upon BEAT-MEV by constructing an efficient CCA-secure batch threshold encryption scheme with *silent* setup and provides engineering-level optimizations that reduce the communication and computation costs of BEAT-MEV. Along with [19], BEAST-MEV also reduces the quadratic-sized public parameters of BEAT-MEV to linear by using a q-type assumption for the puncturable key-homomorphic PRF. On the BEAT-MEV line of work, Agarwal et al. [3] give a weighted batch threshold encryption scheme without virtualization. They also reduce the computational cost of decryption from quadratic to quasi-linear number of pairings. Gong, Waters, Wee, and Wu [22] obtain the first selectively-secure batch IBE scheme based on elliptic curves in the plain model under a q-type assumption. Xiong, Zhang, and Chen [32] present a lattice-based batch IBE scheme in the plain model with security based on ℓ-succinct LWE.

New Work. After a first manuscript of the paper with Section 4 was made public, two new works [4,25], called BTX and Simple BTE, emerged, proposing a new index-independent, epochless, and concretely efficient batch (threshold) encryption scheme with $O(\ell)$-sized public parameters, $O(1)$-sized pre-decryption key and ciphertexts. We include comparison with these new works in Table 1. Notably, their construction achieves a ciphertext size of $|\mathbb{G}_1| + |\mathbb{G}_T|$ (excluding the NIZK proof). Inspired by these results, we optimized our scheme to obtain smaller ciphertexts of size $|\mathbb{G}_1| + |\mathbb{G}_T|$ (compared to $2|\mathbb{G}_1| + |\mathbb{G}_T|$ sized ciphertexts of Section 4) at the expanse of a larger, but still $O(\ell)$-sized CRS. The optimized scheme is documented in Section 5.

1.1 Applications

There are several applications enabled by batch threshold encryption.

1.1.1 Encrypted Mempools

Transactions sent to a blockchain typically remain observable as part of a public "mempool" until being included by a block. This allows exploits such as front-running that extract value at the sender's expense, an issue called maximal extractable value (MEV) [18]. Encrypted mempools aim to mitigate MEV by keeping transactions confidential prior to inclusion [26,7]. Concretely, the sender encrypts the transaction under some public key and submits only the ciphertext to the mempool. The corresponding secret key is distributed (*e.g.*, via threshold sharing) among the blockchain validators. After a block is committed as part of the blockchain, validators release decryption shares of the transactions within that block. This ensures that transaction contents become visible only after they are irreversibly recorded on-chain.

A major challenge for encrypted mempools is communication complexity. With conventional public key encryption, decryption of ℓ transactions in a block would require each of the N blockchain validators to publish ℓ decryption shares, one per ciphertext. This implies a communication complexity of $O(N\ell)$ per block. Batch threshold encryption addresses this by having each validator publish the share of a succinct pre-decryption key. Then, all ℓ transactions can be decrypted upon combining the N shares of the pre-decryption key, reducing the communication complexity to $O(N)$.

Encrypted mempools were a major motivation for the earlier epoch-based schemes, where block height would play the role of an epoch. The sender would encrypt its transaction with respect to the epoch of a block expected to include the transaction. If the anticipated epoch is incorrect, the ciphertext must be encrypted and submitted again. Our epochless and index-independent design solves this issue by removing epochs and making censorship attacks as expensive as in a blockchain without encrypted mempools.

1.1.2 Time-lock Encrypted Storage

First formalized by Agarwal et al. in [2], time-lock encrypted storage enables users to post encrypted values to the blockchain to be decrypted at a specific future time. Unlike prior time-lock encryption schemes, it provides secrecy for the values that were not included by the blockchain, a crucial requirement for many on-chain applications such as sealed-bid auctions, on-chain contests for machine learning models, on-chain elections, and blockchain governance. Their construction provides this feature by combining an epoch-based batch threshold encryption scheme [5] with a time-release threshold IBE scheme. As such, our epochless and censorship-free construction can help improve the usability of these applications by removing epochs and mitigating censorship attacks (*e.g.*, in sealed-bid auctions).

2 Technical Overview

We begin by providing a detailed overview of our non-threshold batch encryption (BE) construction, which we build in multiple stages. First, in Section 2.1, we review the techniques used in [24] to build a threshold encryption scheme. Using these techniques as inspiration, in Section 2.2, we showcase an *index-dependent* BE scheme, intended as a warm-up towards our final construction. We subsequently *combine* this index-dependent scheme with ideas from threshold encryption to obtain our final index-independent BE construction in Section 2.3. In Section 2.6, we describe how to thresholdize our scheme to obtain batch threshold encryption. Finally, in Section 2.7, we show how to shrink the ciphertext to only two group elements.

2.1 Overview of the Threshold Encryption Construction in [24]

Let $\mathbb{F}$ be a finite field of prime order p, and for any $i \in \mathbb{Z}_p$, define $p_i(X) \in \mathbb{F}[X]$ as the rational function, $p_i(X) \coloneqq 1/(X+i)$. The construction is based on the following observations formalized by Lemma 1 in Section 3:

Observation 1. Product of rational functions with linear denominators ("partial fractions") can be expressed as a linear combination of the same partial fractions with easy-to-compute coefficients:

$$p_i(X) \cdot p_j(X) = \frac{1}{j-i}(p_i(X) - p_j(X)) \quad \text{for} \quad i \neq j$$

Observation 2. For a given set $S \subseteq \mathbb{F}$ and arbitrary $(c_i)_{i \in S} \in \mathbb{F}^S$, the following is a set of linearly independent rational functions:

$$\left\{ \left(\sum_{i \in S} c_i p_i(X) + \sum_{i \in S} c_i \frac{1}{j-i} \right) \cdot p_j(X) \right\}_{j \in \mathbb{F} \setminus S}$$

In the following description, we focus on a version of broadcast encryption, in which we set the decryption threshold to 1 and fix the set of parties authorized to decrypt as $\{1, \ldots, k\}$ where the total set of parties is $\{1, \ldots, n\}$. The construction uses a bilinear group[4] of prime order p, $\mathcal{G} \coloneqq (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$, and the public key for party i is $[p_i(x)]_1 \in \mathbb{G}_1$, while the secret key is $[p_i(x)]_2 \in \mathbb{G}_2$ for a uniformly random $x \xleftarrow{\$} \mathbb{F}$.

Additionally, the public parameters contain 2 $\mathbb{G}_1$ elements that will be used during encryption[5], namely $[p_{-1}(x)]_1$ and $[p_0(x)]_1$, and $(n-k)$ $\mathbb{G}_2$ elements, $([p_i(x)]_2)_{i=n+1}^{2n-k}$, that will be used during decryption. These additional elements act as "dummy" secret keys, and do not correspond to real parties.

To encrypt a message $\mathsf{m} \in \mathbb{G}_T$, the encryptor samples $r \xleftarrow{\$} \mathbb{Z}_p$ and computes the following ciphertext:

$$\mathsf{ct} = \left([r]_1,\ r \cdot \left(\sum_{i=k+1}^{n} [p_i(x)]_1 + [p_{-1}(x)]_1 + [p_0(x)]_1 \right),\ r \cdot [p_0(x)]_T + \mathsf{m} \right)$$

The second component of the ciphertext essentially defines a linear system scaled by r over a $(n-k+2)$-sized subset of partial fractions in $\mathbb{G}_1$, namely the subset corresponding to (i) the complement of the authorized set, *i.e.*, the *unauthorized set* $\{k+1, \ldots, n\}$, of size $n-k$, (ii) $p_{-1}(x)$, and (iii) $p_0(x)$. The third component is the message m one-time padded by one of the partial fractions scaled by r, namely $r \cdot [p_0(x)]_T$.

Now, a party $j \in \{1, \ldots, k\}$ can decrypt m by pairing the second component of the ciphertext with its secret key $[p_j(x)]_2$:

$$e\left(r \cdot \left(\sum_{i=k+1}^{n} [p_i(x)]_1 + [p_{-1}(x)]_1 + [p_0(x)]_1 \right), [p_j(x)]_2 \right)$$

[4] Here, we use additive and bracket notation for groups. Please refer to the preliminaries in Section 3 for more details.

[5] $[p_{-1}(x)]_1$ is in fact not necessary for the simplified construction presented here, but is important for our final BE construction.

$$= \left[r \left(\sum_{i=k+1}^{n} p_i(x) + p_{-1}(x) + p_0(x) \right) p_j(x) \right]_T$$

Using Observation 1, we can decompose the product of linear partial fractions as

$$r \cdot \left[\sum_{i=k+1}^{n} \frac{1}{j-i}(p_i(x) - p_j(x)) + \frac{1}{j+1}(p_{-1}(x) - p_j(x)) + \frac{1}{j}(p_0(x) - p_j(x)) \right]_T$$

Besides its secret key, party j pairs the second component of the ciphertext with $[1]_2$ and the $(n-k)$ $\mathbb{G}_2$ elements in the public parameters to obtain $n-k+1$ $\mathbb{G}_T$ elements, corresponding to $n-k+1$ more equations. Each of these equations is linearly independent by Observation 2. Therefore, in total, party j obtains $(n-k+2)$ group elements corresponding to a linearly-independent set of $(n-k+2)$ equations over $(n-k+2)$ variables, which defines a full-rank linear system, and thus it can solve for the one-time pad $[p_0(x)]_T$ and decrypt the message.

While an unauthorized party $j' \notin \{1, \ldots, k\}$ can still recover $n-k+1$ equations using the public parameters and ciphertext, it cannot use its secret key $[p_{j'}(x)]_2$ to compute the first equation. This is because pairing the second component of the ciphertext with $[p_{j'}(x)]_2$ results in a multiple of a *quadratic fraction*, *i.e.*, $[r \cdot (p_{j'}(x))^2]_T$, and is thus not a linear equation of (linear) fractions. Formally, the scheme is proven secure using a new q-type assumption q-$\mathsf{RankDef}$ [24, Assumption 3], which captures this intuition and is shown to hold in the Generic Bilinear Group Model (GBGM).

2.2 Starting Point: Index-Dependent Batch Encryption

We next present an index-dependent BE construction as a building block to our full construction. Here, encryption takes an additional *index* input $i \in [\ell]$, which denotes the sender-chosen algebraic slot for the ciphertext. Intuitively, the scheme uses quadratic fractions as one-time pad for encrypting messages. For any $i \in \mathbb{Z}_p$, let $q_i(X) \in \mathbb{F}[X]$ be the quadratic fraction, $q_i(X) \coloneqq p_i^2(X) = 1/(X+i)^2$.

The setup algorithm samples a uniformly random $x \xleftarrow{\$} \mathbb{Z}_p$ and publishes as the encryption key the following set of fractions in $\mathbb{G}_1$ and $\mathbb{G}_T$:

$$\mathsf{ek} \coloneqq \{([p_i(x)]_2, [q_i(x)]_T)\}_{i \in [\ell]}$$

The secret key sk is set to x, and the decryption key dk is $\perp$.

To encrypt a message $\mathsf{m} \in \mathbb{G}_T$ to an index i, the encryptor samples $r \xleftarrow{\$} \mathbb{Z}_p$ and generates the ciphertext: $\mathsf{ct} = ([r]_1, \mathsf{m} + r \cdot [q_i(x)]_T)$. Let $\mathsf{ct}[i]$ be the i-th component of the tuple ct. Informally, $r \cdot [q_i(x)]_T$ serves as a one-time pad for m, while the first element of the ciphertext is the encryption randomness, represented in $\mathbb{G}_1$, which will be used to link the pre-decryption key to the ciphertext ct at index i, so that the one-time pad can be recovered.

Given ℓ ciphertexts $(\mathsf{ct}_i)_{i \in [\ell]}$, one for each index, the goal of pre-decryption is to generate a succinct pre-decryption key sbk, which would allow each ciphertext ct_i to be decrypted without sk, while maintaining the security of ciphertexts not included in the batch. We notice that we can take a linear combination of the fractions $p_i(x)$ to achieve a constant-size (only one group element) pre-decryption key; such that each ciphertext in the batch can be decrypted using the partial fraction decomposition technique. More specifically, the pre-decryption algorithm, equipped with the secret key $\mathsf{sk} = x$, generates:

$$\mathsf{sbk} \coloneqq \sum_{i \in [\ell]} p_i(x) \cdot \mathsf{ct}_i[1] = \sum_{i \in [\ell]} p_i(x) \cdot [r_i]_1 \in \mathbb{G}_1$$

Now, given sbk and the ciphertext, the decryptor can decrypt any ciphertext ct_j, $j \in \{1, \ldots, \ell\}$, in the batch. For this purpose, it first pairs $[p_j(x)]_2$ with sbk:

$$e(\mathsf{sbk}, [p_j(x)]_2) = e\left(\sum_{i\in[\ell]} p_i(x) \cdot [r_i]_1, [p_j(x)]_2\right) = \sum_{i\in[\ell]} r_i [p_j(x)p_i(x)]_T$$

Here, for each $i \neq j$, using Observation 1, we can decompose $p_i(X) \cdot p_j(X)$ into

$$\frac{1}{j-i}(p_i(X) - p_j(X))$$

and thus obtain

$$e(\mathsf{sbk}, [p_j(x)]_2) = \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{r_i}{j-i}[p_i(x) - p_j(x)]_T + r_j \cdot [q_j(x)]_T$$

Next, it computes the sum of linear partial fractions in the equation above via $\ell - 1$ pairings:

$$\left[\sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{r_i}{j-i}(p_i(x) - p_j(x))\right]_T = \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}[r_i \cdot (p_i(x) - p_j(x))]_T$$

$$= \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i} e\big([r_i]_1, [p_i(x) - p_j(x)]_2\big) = \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i} e\big(\mathsf{ct}_i[1], [p_i(x)]_2 - [p_j(x)]_2\big)$$

By subtracting this sum from the pairing of sbk and $[p_j(x)]_2$, the decryptor exposes the quadratic term, *i.e.*, the one-time pad. Finally, it recovers the message by subtracting the quadratic term from $\mathsf{ct}_j[2]$:

$$\mathsf{m}_j := \mathsf{ct}_j[2] - \left(e(\mathsf{sbk}, [p_j(x)]_2) - \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i} e\big(\mathsf{ct}_i[1], [p_i(x)]_2 - [p_j(x)]_2\big)\right)$$

If two selected ciphertexts use the same index, the decryption algorithm cannot obtain their one-time pads separately, implying that the scheme is index-dependent. However, the scheme is useful for illustrating how a single group element $\sum_i p_i(x)\, ct_i[1]$ can compactly encode a whole batch. While it also improves on the state-of-the-art index-dependent BE schemes with its short ciphertexts (2 group elements), our goal is to obtain an index-independent construction, which we describe next.

2.3 Removing Index Dependence

To remove index dependence, we convert encryption-time indices into decryption-time batch positions determined by the pre-decryption algorithm. For this purpose, we combine the constructions in Sections 2.1 and 2.2 by treating the indices $i \in [\ell]$ as the parties in the threshold encryption construction and associating a "public" and "secret" key for each possible index. More specifically, the setup algorithm outputs $[p_0(x)]_1$ and $[p_{-1}(x)]_1$ along with $p_i(x)$ in both $\mathbb{G}_1$ and $\mathbb{G}_2$ for $i \in [\ell]$.

We no longer require the quadratic fractions in $\mathbb{G}_T$, as we will instead use a linear partial fraction for the one-time pad. Then, the modified encryption key ek becomes $([p_i(x)]_1)_{i=-1,\ldots,\ell}$[6], decryption key dk is $([p_i(x)]_2)_{i=1,\ldots,\ell}$, and the secret key sk is x. Interestingly, setup publishes $p_i(x)$ in both $\mathbb{G}_1$ and $\mathbb{G}_2$, which in the context of the threshold encryption construction outlined above is revealing both the public and secret keys for each index.

The ciphertexts have the same form as in the threshold encryption construction:

$$\mathsf{ct} = \left([r]_1, r \cdot \left(\sum_{i=-1}^{\ell} [p_i(x)]_1\right), r \cdot e([p_0(x)]_1, [1]_2) + \mathsf{m}\right)$$

Again, $p_0(x)$ is used as a one-time pad, and $p_{-1}(x)$ in the second component acts as a dummy fraction to introduce another variable in the linear system.

Note that the "unauthorized" set implied by the ciphertext contains *every* index $i \in \{1, \ldots, \ell\}$. Intuitively, this means that the "secret" keys $[p_j(x)]_2$ for the indices cannot be used to decrypt the ciphertext, due to the quadratic fraction $[q_j(x)]_T$ that appears when the ciphertext is paired with any $[p_j(x)]_2$. However, in the index-dependent BE construction, we showed how to peel off the quadratic fractions that appear during decryption. It turns out that the same approach is sufficient for the full index-independent construction as well. So, the pre-decryption algorithm proceeds as above:

$$\mathsf{sbk} \coloneqq \sum_{i \in [\ell]} p_i(x) \cdot \mathsf{ct}_i[1] = \sum_{i \in [\ell]} p_i(x) \cdot [r_i]_1 \in \mathbb{G}_1 \,.$$

To decrypt a ciphertext ct_j, the decryptor first follows the steps in Section 2.2 and pairs sbk with $[p_j(x)]_2$:

$$e(\mathsf{sbk}, [p_j(x)]_2) = \left[r_j q_j(x) + \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{r_i}{j-i} (p_i(x) - p_j(x)) \right]_T ,$$

where the equation follows from the same analysis in Section 2.2, which in turn is based on partial fraction decomposition (Observation 1). Next, it computes the linear partial fractions within the expression above as in Section 2.2, again with $\ell - 1$ pairings:

$$\begin{aligned} \left[\sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{r_i}{j-i} (p_i(x) - p_j(x)) \right]_T &= \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{1}{j-i} [r_i \cdot (p_i(x) - p_j(x))]_T \\ = \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{1}{j-i} e\big([r_i]_1, [p_i(x) - p_j(x)]_2\big) &= \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{1}{j-i} e\big(\mathsf{ct}_i[1], [p_i(x) - p_j(x)]_2\big) \,. \end{aligned} \tag{1}$$

Finally, it subtracts these partial fractions to expose the quadratic fraction:

$$[r_j q_j(x)]_T = e(\mathsf{sbk}, [p_j(x)]_2) - \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{1}{j-i} e\big(\mathsf{ct}_i[1], [p_i(x) - p_j(x)]_2\big) \,. \tag{2}$$

[6] In fact, we don't need to publish each partial fraction individually and only need to publish the sum along with $[p_0(x)]_1$.

Having obtained the quadratic fractions $[r_j q_j(x)]_T$, the decryptor next proceeds with the decryption steps as in Section 2.1, and pairs $\mathsf{ct}_j[2]$ with the "secret" key $[p_j(x)]_2$:

$$e\big(\mathsf{ct}_j[2], [p_j(x)]_2\big) = e\left(r_j \cdot \left(\sum_{i=-1}^{\ell} [p_i(x)]_1\right), [p_j(x)]_2\right)$$
$$= \left[r_j q_j(x) + \sum_{\substack{i=-1 \\ i\neq j}}^{\ell} r_j p_i(x) p_j(x)\right]_T = \left[r_j q_j(x) + \sum_{\substack{i=-1 \\ i\neq j}}^{\ell} \frac{r_j}{j-i}(p_i(x) - p_j(x))\right]_T ,$$

where it can now remove the quadratic fraction, $[r_j q_j(x)]_T$. Finally, it also peels off the multiples of $p_i(x)$ for all $i \in [\ell]$ by subtracting the sum

$$e\left(\mathsf{ct}_j[1], \sum_{\substack{i\in[\ell] \\ i\neq j}} \frac{1}{j-i}[p_i(x)]_2 - \left(\sum_{\substack{i=-1 \\ i\neq j}}^{\ell} \frac{1}{j-i}\right)[p_j(x)]_2\right)$$

so that only the following terms remain:

$$r_j \cdot \left(\frac{1}{j+1}[p_{-1}(x)]_T + \frac{1}{j}[p_0(x)]_T\right). \tag{3}$$

To solve for $r_j[p_0(x)]_T$ and decrypt the ciphertext, it must obtain one more equation from the ciphertext itself by pairing the second component with $[1]_2$ and subsequently peeling off the terms $r_j \cdot p_i(x)$ similar to above. This gives

$$e\big(\mathsf{ct}_j[2], [1]_2\big) - e\left(\mathsf{ct}_j[1], \sum_{i\in[\ell]} [p_i(x)]_2\right) = r_j \cdot ([p_{-1}(x)]_T + [p_0(x)]_T). \tag{4}$$

Using Equations (3) and (4), we can solve for $R_j \coloneqq r_j \cdot [p_0(x)]_T$, which is precisely the one-time pad. So we obtain $\mathsf{m}_j \coloneqq \mathsf{ct}_j[3] - R_j$. We remark that the "peeling" of $[r_j p_i(x)]_T$ is effectively solving the linear system with the two derived equations with an additional ℓ equations of the form $[r_j p_i(x)]_T$.

2.4 Amortizing Pairings and Group Operations

The construction outlined above requires $\ell + 4$ pairings per ciphertext resulting in a total of $O(\ell^2)$ pairings to decrypt the whole batch of ℓ ciphertexts. By careful algebraic manipulation, we can reduce this to a total of only 5ℓ pairings by precomputing pairing equations reused across decryptions in the same batch. In more detail, observe that for each $j \in [\ell]$, we require $\ell - 1$ pairings to compute

$$\left[\sum_{\substack{i\in[\ell] \\ i\neq j}} \frac{r_i}{j-i}(p_i(x) - p_j(x))\right]_T = \sum_{\substack{i\in[\ell] \\ i\neq j}} \frac{1}{j-i}[r_i p_i(x)]_T$$

$$- e\left(\sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\cdot \mathsf{ct}_i[1], [p_j(x)]_2\right)$$

in Equation (1). Instead, we can precompute $[r_i p_i(x)]_T = e(\mathsf{ct}_i[1], [p_i(x)]_2)$ for each $i \in [\ell]$ and take a linear combination of these values to compute the required sum via target group operations. Furthermore, our scheme is amenable to FFTs, which reduces group operations from quadratic to quasi-linear, *i.e.*, $O(\ell \log \ell)$.

The final optimized pairing equations, and explanation of the FFT technique are presented in the full construction in Section 4.1.2.

2.5 CCA-2 Security

A batch encryption scheme must satisfy the stronger CCA-2 notion of semantic security to prevent attacks such as ciphertext malleability and copying. In other words, given pre-decryption keys for adversarially chosen batches of ciphertexts, the adversary should still be unable to distinguish the ciphertexts for adversarially chosen messages. The adversary is allowed to query for pre-decryption keys on any ciphertext except the challenge. We formalize this as a game in Figure 2.

To achieve CCA-2 security, we employ *non-interactive zero-knowledge proofs* (NIZKs), which prove that the ciphertext is correctly formed, and the encryptor knows the ciphertext randomness r. Crucially, the NIZK statement contains the *entire* ciphertext, including the component that carries the masked message. The algebraic relation leaves that component unconstrained; it enters the proof only as a tag bound by the Fiat–Shamir hash. This ensures that modifying *any* component of a ciphertext invalidates its proof and thus prevents malleability attacks. When the adversary asks for a ciphertext on one of its challenge messages, the challenger must simulate a NIZK proof for the returned ciphertext. Therefore, we also require the NIZK to satisfy the stronger property of *simulation-extractability*.

Additionally, due to technical reasons, the extractor for the NIZK must be *straightline*. (See [29] for a detailed discussion.) We introduce the requisite definitions in Section 3, and our simulation-extractable NIZK construction in Section 4.3. We prove its straightline simulation-extractability in the Algebraic Group Model (AGM) and Random Oracle Model (ROM). A similar use of simulation-extractable NIZKs for CCA-2 security is seen in [14]. We discuss potential tradeoffs on relaxing the idealized model in which we prove security in Section 2.8.

2.6 Thresholdizing Batch Encryption

We thresholdize the BE construction via Shamir secret sharing. Each $p_i(x) \in \mathbb{F}$ is t-of-N secret shared into $(s_1^i, \ldots, s_N^i)$, and the secret key sk_j of party $j \in [N]$ consists of the secret shares s_j^i of $p_i(x)$ for each $i \in [\ell]$. To generate its pre-decryption key share, each party $j \in [N]$ computes

$$\mathsf{sbk}_j := \sum_{i\in[\ell]} s_j^i \cdot \mathsf{ct}_i[1]$$

and the pre-decryption key is found by addition of the shares from a subset T of t parties:

$$\mathsf{sbk} := \sum_{j\in T} \lambda_j \cdot \mathsf{sbk}_j = \sum_{j\in T} \lambda_j \sum_{i\in[\ell]} s_j^i \cdot \mathsf{ct}_i[1]$$

$$= \sum_{i\in[\ell]} \left(\sum_{j\in T} \lambda_j s_j^i \right) \cdot \mathsf{ct}_i[1] = \sum_{i\in[\ell]} p_i(x) \cdot \mathsf{ct}_i[1],$$

where $\lambda_j \in \mathbb{Z}_p$ is the Lagrange coefficient for party j. Although the size of the secret key is $O(\ell)$, each pre-decryption key share is a single group element. We prove the security and robustness (against malicious parties) of the thresholdized scheme in Section 4.2.

2.7 Shrinking Ciphertext Size

We can reduce the size of the ciphertext by removing the second component and instead, including additional $\mathbb{G}_2$ partial fractions in the decryption key to enable computing Equations (3) and (4) directly.

We motivate the optimized construction by working backwards from the one in the previous section. Recall that our goal is to recover the one-time pad $R_j = r_j \cdot [p_0(x)]_T$ for each $j \in [\ell]$. We were able to do so by computing two linearly-independent group elements as in Equations (3) and (4).

First, we take another look at Equation (4):

$$\begin{aligned} r_j \cdot ([p_{-1}(x)]_T + [p_0(x)]_T) &= e([r_j]_1, [p_{-1}(x) + p_0(x)]_2) \\ &= e\big(\mathsf{ct}_j[1], [p_{-1}(x) + p_0(x)]_2\big) \ . \end{aligned}$$

So, if we can include $[p_{-1}(x) + p_0(x)]_2$ in the decryption key without compromising security, we can directly compute Equation (4).

Next, we demonstrate another method to compute Equation (3). We proceed with the same sbk as before. In the previous section, we showed how to isolate $[r_j q_j(x)]_T$, which we later used to remove the quadratic term from the pairing equation derived from $\mathsf{ct}_j[2]$. Instead, we can simply include "masked" group elements in the decryption key as follows:

$$[w_j(x)]_2 \coloneqq \left[q_j(x) + \frac{1}{j+1} p_{-1}(x) + \frac{1}{j} p_0(x) \right]_2 \text{ for each } j \in [\ell].$$

Then, to recover Equation (3), we can pair $\mathsf{ct}_j[1] = [r_j]_1$ with $[w_j(x)]_2$ and remove the quadratic q_j:

$$\begin{aligned} e(\mathsf{ct}_j[1], [w_j(x)]_2) - [r_j q_j(x)]_T &= r_j \cdot \left[q_j(x) + \frac{1}{j+1} p_{-1}(x) + \frac{1}{j} p_0(x) \right]_T \\ &\quad - [r_j q_j(x)]_T \\ &= r_j \cdot \left(\frac{1}{j+1} [p_{-1}(x)]_T + \frac{1}{j} [p_0(x)]_T \right), \end{aligned}$$

where $[r_j q_j(x)]_T$ is computed as in Equation (2).

Thus, if we increase the decryption key by $(\ell + 1)$ $\mathbb{G}_2$ elements, we can obtain a ciphertext size of only $|\mathbb{G}_1| + |\mathbb{G}_T|$. Note that we no longer need the sum of $\mathbb{G}_1$ partial fractions during encryption, so the encryption key also shrinks by one $\mathbb{G}_1$ element. We prove security using a variant of the assumption used in the previous construction. We show that the new assumption also holds in Generic Bilinear Group Model. We present our optimized batch encryption construction with shorter ciphertexts in Section 5.

2.8 Idealized Model Tradeoffs

To ensure the straightline simulation-extractability of the NIZK, we prove the security of our schemes in the Algebraic Group Model (AGM) and Random Oracle Model (ROM). It is also possible to prove straightline simulation-extractability only in ROM by applying the Fischlin transform [27] to our NIZK (Σ-protocol) in Section 4.3, albeit at the expense of larger proofs. We leave it as future work to design a batch encryption scheme in the plain model.

3 Preliminaries

In this section we describe our notations and explain the necessary background that we use in our constructions.

3.1 Notation

We denote the security parameter by λ, the set of natural numbers and integers by $\mathbb{N}$ and $\mathbb{Z}$, the set of integers modulo a prime p by $\mathbb{Z}_p$, and the (prime) order p finite field by $\mathbb{F}_p$, which we may write simply as $\mathbb{F}$. We assume that $p = \Omega(2^\lambda)$. For any $n \in \mathbb{N}$, let $[n] \coloneqq \{1, \ldots, n\}$ and $(a_i)_{i \in S} \coloneqq (a_{i_1}, \ldots, a_{i_t})$ for $S = (i_1, \ldots, i_t) \subseteq [n]$. We denote a negligible function in λ (*i.e.*, $o(\lambda^{-c})$ for all $c \in \mathbb{N}$) as $\mathsf{negl}(\lambda)$. Let "$\stackrel{?}{=}$" be a binary operator that outputs 1 if its operands are equal (and well-defined) and 0 otherwise. We use "$\wedge$" for the logical AND operator. Let $x \xleftarrow{\$} \mathcal{X}$ denote the uniform random variable over a finite set $\mathcal{X}$, and for a randomized algorithm $\mathcal{A}$, $y \leftarrow \mathcal{A}(x)$ denotes the random variable obtained by running $\mathcal{A}$ on input x. We denote computationally indistinguishable distributions and games by $\approx_c$.

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathrm{T}$ denote cyclic (additive) groups of prime order p that admit an efficient (non-degenerate) bilinear pairing map, $e \colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_\mathrm{T}$. Let g_1, g_2 be generators of groups $\mathbb{G}_1, \mathbb{G}_2$ respectively and $g_T \coloneqq e(g_1, g_2)$. We call $\mathcal{G} \coloneqq (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_\mathrm{T}, g_1, g_2, e)$ a prime-order bilinear group. We assume that there exists an efficient, randomized algorithm $\mathsf{BGrpGen}(1^\lambda)$ that outputs descriptions of $\mathcal{G}$. We denote the group element $x \cdot g_i$ by $[x]_i$ for $i = 1, 2, T$ (*e.g.*, $[1]_1 = g_1$).

3.2 Partial Fraction Decomposition

We restate [24, Lemma 3] below.

Lemma 1 (Linear Independence of Partial Fraction Products). *Let $(c_i)_{i \in [n]} \subseteq \mathbb{F}$ be arbitrary field elements. Let $(a_i)_{i \in [n]} \subseteq \mathbb{F}$ be a set of distinct elements from $\mathbb{F}$, and let $b_1, \ldots, b_m \in \mathbb{F}$ be distinct elements such that $\{a_i\}_{i \in [n]} \cap \{b_1, \ldots, b_m\} = \varnothing$. Then, for $j = 1, \ldots, m$,*

$$\left(\sum_{i=1}^{n} \frac{c_i}{X + a_i} + \sum_{i=1}^{n} \frac{c_i}{b_j - a_i} \right) \cdot \frac{1}{X + b_j} = \sum_{i=1}^{n} \frac{c_i}{b_j - a_i} \cdot \frac{1}{X + a_i},$$

and these rational functions are linearly-independent over $\mathbb{F}$ provided that $m \leq n$.

3.3 Batch Encryption

The following definitions for batch encryption and batch threshold encryption are based on [14, Definitions 3.2, 3.3 and 3.4].

Definition 1 (Batch Encryption). *A batch encryption (BE) scheme instantiated with a maximum batch size of $\ell \in \mathbb{N}$ and message space of $\mathcal{M}$ consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda, 1^\ell) \to (\mathsf{ek}, \mathsf{sk}, \mathsf{dk})$*. The setup algorithm takes the security parameter λ and the batch size ℓ, and outputs a public encryption key* ek*, a secret key* sk*, and a public decryption key* dk*.*
- $\mathsf{Enc}(\mathsf{ek}, m) \to \mathsf{ct}$*: The encryption algorithm takes the encryption key* ek *and a message $m \in \mathcal{M}$, and outputs a ciphertext* ct*.*
- $\mathsf{PreDec}(\mathsf{sk}, (\mathsf{ct}_i)_{i\in[\ell]}) \to \mathsf{sbk}$*: The (deterministic) pre-decryption algorithm takes a secret key* sk *and ℓ ciphertexts, and outputs a pre-decryption key* sbk *or $\perp$.*
- $\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) \to (\mathsf{m}_i)_{i\in[\ell]}$*: The (deterministic) decryption algorithm takes the decryption key* dk *and a pre-decryption key* sbk *and ℓ ciphertexts, and outputs the corresponding messages.*

Note that smaller batches can be handled by adding dummy ciphertexts to the batch.

A BE scheme must be correct and secure against chosen-ciphertext attacks as formalized below. Let $\mathsf{BE} = (\mathsf{Setup}, \mathsf{Enc}, \mathsf{PreDec}, \mathsf{Dec})$ be a BE scheme.

Definition 2 (Correctness of BE). *We say* BE *is correct or satisfies correctness if for all $\lambda, \ell \in \mathbb{N}$, and $(\mathsf{m}_i)_{i\in[\ell]} \in \mathcal{M}^\ell$, the following probability is greater than $1 - \mathsf{negl}(\lambda)$:*

$$\Pr\left[\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) = (\mathsf{m}_i)_{i\in[\ell]} \;\middle|\; \begin{array}{c} (\mathsf{ek}, \mathsf{sk}, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell) \\ \mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{ek}, \mathsf{m}_i)\ \forall i \in [\ell] \\ \mathsf{sbk} \leftarrow \mathsf{PreDec}(\mathsf{sk}, (\mathsf{ct}_i)_{i\in[\ell]}) \end{array}\right]$$

Definition 3 (B-IND-CCA Security of BE). BE *is* B-IND-CCA*-secure, if for all $\ell = \mathsf{poly}(\lambda)$ and PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$, such that*

$$\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}} := \left|\Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell) = 1] - \frac{1}{2}\right| < \mathsf{negl}(\lambda)$$

where $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell)$ is defined in Figure 2 with respect to the adversary $\mathcal{A}$ and the batch encryption scheme BE*.*

B-IND-CCA can be viewed as semantic security against chosen ciphertext attacks modified to include batch decryption queries.

3.4 Batch Threshold Encryption

Definition 4 (Batch Threshold Encryption). *A batch threshold encryption (BTE) scheme instantiated with $N \in \mathbb{N}$ parties, a threshold of $t \in [N]$, maximum batch size of $\ell \in \mathbb{N}$, and message space of $\mathcal{M}$ consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda, 1^\ell, 1^N, 1^t) \to (\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk})$*. The setup algorithm takes the security parameter λ, the maximum batch size ℓ, the number of parties N, and the threshold t, and outputs a public encryption key* ek*, a public decryption key* dk*, and N secret keys* $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N)$*.*

- $\mathsf{Enc}(\mathsf{ek}, m) \to \mathsf{ct}$: *The encryption algorithm takes the encryption key* ek *and a message* $m \in \mathcal{M}$*, and outputs a ciphertext* ct*.*
- $\mathsf{PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i)_{i\in[\ell]}) \to \mathsf{sbk}_j$: *The (deterministic) pre-decryption algorithm takes a secret key* sk_j *for a party* $j \in [N]$ *and* ℓ *ciphertexts, and outputs a pre-decryption key share* sbk_j *or* $\perp$*.*
- $\mathsf{Combine}(\mathsf{dk}, T \subseteq [N], (\mathsf{sbk}_j)_{j\in T}) \to \mathsf{sbk}$: *The (deterministic) combiner algorithm takes a subset* $T \subseteq [N]$*, and the pre-decryption key shares of these* $|T|$ *parties, and outputs a pre-decryption key* sbk *or* $\perp$*.*
- $\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) \to (\mathsf{m}_i)_{i\in[\ell]}$: *The (deterministic) decryption algorithm takes the decryption key* dk*, a pre-decryption key* sbk *and* ℓ *ciphertexts, and outputs the corresponding messages.*

A BTE scheme must be correct and secure against chosen-ciphertext attacks (CCA) as formalized below. Let $\mathsf{BTE} = (\mathsf{Setup}, \mathsf{Enc}, \mathsf{PreDec}, \mathsf{Combine}, \mathsf{Dec})$ be a BTE scheme.

Definition 5 (Correctness of BTE). *We say* BTE *is correct or satisfies correctness if for all* $\lambda, \ell \in \mathbb{N}$*,* $T \subseteq [N]$ *such that* $|T| = t$*, and* $(\mathsf{m}_i)_{i\in[\ell]} \in \mathcal{M}^\ell$*, the following probability is greater than* $1 - \mathsf{negl}(\lambda)$*.*

$$\Pr\left[\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) = (\mathsf{m}_i)_{i\in[\ell]} \;\middle|\; \begin{array}{c} (\mathsf{ek}, (\mathsf{sk}_j)_{j\in[N]}, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, 1^N, 1^t) \\ \mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{ek}, \mathsf{m}_i)\ \forall i \in [\ell] \\ \mathsf{sbk}_j \leftarrow \mathsf{PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i)_{i\in[\ell]})\ \forall j \in T \\ \mathsf{sbk} \leftarrow \mathsf{Combine}(\mathsf{dk}, T, (\mathsf{sbk}_j)_{j\in T}) \end{array}\right]$$

Definition 6 (B-IND-CCA Security of BTE). BTE *is* B-IND-CCA*-secure, if for all* $\ell, N = \mathsf{poly}(\lambda)$*,* $t \le N$*, and PPT adversaries* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$*, such that*

$$\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A},\ell,N,t} := \left|\Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t) = 1] - \frac{1}{2}\right| < \mathsf{negl}(\lambda)$$

where $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t)$ *is defined in Figure 2 with respect to the adversary* $\mathcal{A}$ *and the batch threshold encryption scheme* BTE*.*

Note that Figure 2 defines security with respect to a static adversary. We leave it as future work to extend security to adaptive adversaries.

To ensure robustness against malicious parties, we require a BTE scheme to satisfy *accurate blaming* and *consistency*. In a robust scheme, the combiner algorithm outputs a set of blamed parties, whose pre-decryption shares are not correctly formed. We adapt the definitions of accurate blaming and consistency for a threshold encryption scheme as stated in [12, Definitions 22.13 & 22.14] to the setting of BTE below.

Definition 7 (Accurate Blaming). BTE *ensures accurate blaming if for all possible parameters* $(\mathsf{ek}, (\mathsf{sk}_j)_{j\in[N]}, \mathsf{dk})$*, all ciphertext batches* $(\mathsf{ct}_i)_{i\in[\ell]}$*, all* t*-size subsets* $T \subseteq [N]$*, and all collections of pre-decryption keys* $\left(\mathsf{sbk}_j\right)_{j\in[N]}$*,*

$$\mathsf{Combine}(\mathsf{dk}, T, \left(\mathsf{sbk}_j\right)_{j\in T}) = \mathsf{Blame}(T^*)$$

implies that

$$\Pr\left[\mathsf{PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i)_{i\in[\ell]}) = \mathsf{sbk}_j\right] = 0 \quad \textit{for all } j \in T^*.$$

Definition 8 (Consistency). BTE *is consistent if for all* $\ell, N = \mathsf{poly}(\lambda)$, $t \leq N$, *and PPT adversaries* $\mathcal{A}$, *there exists a negligible function* $\mathsf{negl}(\cdot)$, *such that*

$$\mathsf{Adv}^{\mathsf{Cons}}_{\mathsf{BTE},\mathcal{A},\ell,N,t} := \Pr[\mathsf{Game}^{\mathsf{Cons}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t) = 1] < \mathsf{negl}(\lambda)$$

where $\mathsf{Game}^{\mathsf{Cons}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t)$ *is defined in Figure 1 with respect to the adversary* $\mathcal{A}$ *and the batch threshold encryption scheme* BTE.

$\mathsf{Game}^{\mathsf{Cons}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t)$

1. The challenger Ch takes as input a security parameter λ, a maximum batch size ℓ, a number of parties N, and a threshold size t. It then runs $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, 1^N, 1^t)$ and sends all this data to $\mathcal{A}$.
2. $\mathcal{A}$ outputs
$$\Big((\mathsf{ct}_i)_{i\in[\ell]}, S_1, \{\mathsf{sbk}^1_j\}_{j\in S_1}, S_2, \{\mathsf{sbk}^2_j\}_{j\in S_2}\Big),$$
where $(\mathsf{ct}_i)_{i\in[\ell]}$ is a batch of ciphertexts, $S_1, S_2 \subseteq [N]$ are subsets of size t, and $\{\mathsf{sbk}^1_j\}_{j\in S_1}$ and $\{\mathsf{sbk}^2_j\}_{j\in S_2}$ are pre-decryption key shares.
3. Define
$$\mathsf{Combine}(\mathsf{dk}, S_1, (\mathsf{sbk}^1_j)_{j\in S_1}) = \mathsf{sbk}^1, \text{ where } \mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}^1, (\mathsf{ct}_i)_{i\in[\ell]}) = (\mathsf{m}^1_i)_{i\in[\ell]}$$
$$\mathsf{Combine}(\mathsf{dk}, S_2, (\mathsf{sbk}^2_j)_{j\in S_2}) = \mathsf{sbk}^2, \text{ where } \mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}^2, (\mathsf{ct}_i)_{i\in[\ell]}) = (\mathsf{m}^2_i)_{i\in[\ell]}$$
$\mathcal{A}$ wins the game if $(\mathsf{m}^1_i)_{i\in[\ell]} \neq (\mathsf{m}^2_i)_{i\in[\ell]}$ (and $\mathsf{Combine}$ output valid pre-decryption keys).

Fig. 1: The consistency game for Batch Threshold Encryption.

A BE (or BTE) scheme must be *efficient* in that the size of the pre-decryption key sbk (resp. the pre-decryption key shares sbk_j, $j \in [N]$) is succinct in the maximum batch size ℓ.

3.5 Non-Interactive Zero Knowledge Proofs (NIZKs)

Definition 9 (Non-Interactive Zero Knowledge Proofs). *Let* $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ *be a relation over sets* $\mathcal{X}$ *and* $\mathcal{W}$. *A NIZK scheme for* $\mathcal{R}$ *is a tuple of PPT algorithms* $\mathsf{NIZK} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$,[7] *defined as follows.*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{crs}$*: given the security parameter* λ, *outputs a common reference string* crs. *The algorithms below implicitly take* crs *as input.*
- $\mathsf{Prove}(x, w) \to \pi$*: given a* statement $x \in \mathcal{X}$ *and a* witness $w \in \mathcal{W}$, *outputs a proof* π.
- $\mathsf{Verify}(x, \pi) \to 0/1$*: given a* statement $x \in \mathcal{X}$ *and a* proof π, *outputs 0 or 1.*

A NIZK scheme must be complete, sound and zero-knowledge. Let $\mathcal{L}_\mathcal{R}$ denote the language defined by the relation $\mathcal{R}$.

[7] Later, we will instead have a family of indexed relations to capture public parameters with which the relation is specified.

$\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell)$

Setup: The challenger Ch takes as input a security parameter λ and a maximum batch size ℓ. It runs $(\mathsf{ek}, \mathsf{sk}, \mathsf{dk}) \leftarrow \mathsf{BE.Setup}(1^\lambda, 1^\ell)$ and sends ek and dk to $\mathcal{A}$. The rest of the game proceeds as follows:

Pre-challenge queries: The adversary may issue an arbitrary number of pre-challenge queries for computing pre-decryption keys:

- $\mathcal{A}$ sends a list $(\mathsf{ct}_i)_{i\in[\ell]}$ of ℓ ciphertexts to Ch.
- Ch computes the pre-decryption key $\mathsf{sbk} := \mathsf{BE.PreDec}(\mathsf{sk}, (\mathsf{ct}_i)_{i\in[\ell]})$ and sends sbk to $\mathcal{A}$.

Challenge round: At any point in time, $\mathcal{A}$ may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$ sends two messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathcal{M}$, on which it would like to be challenged.
- Ch samples $b \xleftarrow{\$} \{0,1\}$, computes $\mathsf{ct}_b \leftarrow \mathsf{BE.Enc}(\mathsf{ek}, \mathsf{m}_b)$, and sends ct_b to $\mathcal{A}$.

Post-challenge queries: The adversary can again issue an arbitrary number of queries for computing pre-decryption keys. These queries are answered as the pre-challenge queries, except that if $\mathcal{A}$ sends a list $(\mathsf{ct}_i)_{i\in[\ell]}$ such that $\mathsf{ct}_b \in (\mathsf{ct}_i)_{i\in[\ell]}$, then Ch rejects the query by responding with $\perp$.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0,1\}$. The game then halts with the output $b' \stackrel{?}{=} b$.

$\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t)$

Setup: The challenger Ch takes as input a security parameter λ, a maximum batch size ℓ, a number of parties N, and a threshold size t. It runs $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{BTE.Setup}(1^\lambda, 1^\ell, 1^N, 1^t)$ and sends ek and dk to $\mathcal{A}$. It then receives a subset $S \subseteq [N]$ of size $t-1$ from $\mathcal{A}$ and responds with $(\mathsf{sk}_j)_{j\in S}$. The rest of the game proceeds as follows:

Pre-challenge queries: The adversary may issue an arbitrary number of pre-challenge queries for computing pre-decryption key shares:

- $\mathcal{A}$ sends a list $(\mathsf{ct}_i)_{i\in[\ell]}$ of ℓ ciphertexts to Ch.
- Ch computes the pre-decryption key shares $\mathsf{sbk}_j := \mathsf{BTE.PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i)_{i\in[\ell]})$ for $j \in [N] \setminus S$, and sends them to $\mathcal{A}$.

Challenge round: At any point in time, $\mathcal{A}$ may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$ sends two messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathcal{M}$, on which it would like to be challenged.
- Ch samples $b \xleftarrow{\$} \{0,1\}$, computes $\mathsf{ct}_b \leftarrow \mathsf{BTE.Enc}(\mathsf{ek}, \mathsf{m}_b)$, and sends ct_b to $\mathcal{A}$.

Post-challenge queries: The adversary can again issue an arbitrary number of queries for computing pre-decryption key shares. These queries are answered as the pre-challenge queries, except that if $\mathcal{A}$ sends a list $(\mathsf{ct}_i)_{i\in[\ell]}$ such that $\mathsf{ct}_b \in (\mathsf{ct}_i)_{i\in[\ell]}$, then Ch rejects the query by responding with $\perp$.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0,1\}$. The game then halts with the output $b' \stackrel{?}{=} b$.

Fig. 2: The B-IND-CCA security games for Batch Encryption and Batch Threshold Encryption.

Definition 10 (Completeness). *A NIZK scheme for $\mathcal{R}$ is complete if for all $(x, w) \in \mathcal{R}$, it holds that*

$$\Pr[\mathsf{Verify}(x, \pi) = 1 |\ \pi \leftarrow \mathsf{Prove}(x, w)] \geq 1 - \mathsf{negl}(\lambda)$$

Definition 11 (Soundness). *A NIZK scheme for $\mathcal{R}$ is sound if for all $x \notin \mathcal{L}_{\mathcal{R}}$ and all PPT provers P^*, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\Pr[\mathsf{Verify}(x, \pi) = 1 |\ \pi \leftarrow P^*(x)] \leq \mathsf{negl}(\lambda)$$

Definition 12 (Zero-Knowledge). *A NIZK scheme for $\mathcal{R}$ is zero-knowledge if there exists a PPT simulator Sim such that for all $(x, w) \in \mathcal{R}$, $(x, \mathsf{Sim}(x)) \approx_c (x, \mathsf{Prove}(x, w))$.*

In this work, we require *simulation-extractability* (SE), which implies special soundness and knowledge soundness. Informally, a NIZK satisfies SE if a prover cannot generate new accepting statement–proof pairs (Verify outputs 1) even with access to simulated proofs if it does not *know* a valid witness. In other words, we can construct an extractor that works even using simulated proofs. Formally, a NIZK is an SE-NIZK if it satisfies the following additional property (based on [15, §3.2]):

Definition 13 (Simulation-extractability). *A NIZK scheme NIZK for $\mathcal{R}$ is said to be simulation-extractable or an SE-NIZK if there exists a PPT extractor Ext and a PPT simulator $\mathsf{Sim} = (\mathsf{SimSetup}, \mathsf{SimProve})$ such that for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ for which, the following holds:*

$$\Pr\left[\begin{array}{l|l} \mathsf{Verify}(x^\star, \pi^\star) = 1 \wedge & (\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{td}_{\mathsf{Ext}}) \leftarrow \mathsf{SimSetup}(1^\lambda); \\ (x^\star, \pi^\star) \notin Q \wedge & Q \leftarrow \emptyset; \\ (x^\star, w^\star) \notin \mathcal{R} & (x^\star, \pi^\star) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(\mathsf{crs}); \\ & w^\star \leftarrow \mathsf{Ext}(\mathsf{crs}, \mathsf{td}_{\mathsf{Ext}}, x^\star, \pi^\star) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Here, $\mathcal{O}_{\mathsf{Sim}}(x)$ denotes the simulation oracle that initializes $Q \leftarrow \emptyset$:

$$\pi \leftarrow \mathsf{SimProve}(\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, x);\ Q \leftarrow Q \cup \{(x, \pi)\};\ \textit{return } \pi$$

We denote the probability above by $\mathsf{Adv}^{\mathsf{SE}}_{\mathsf{NIZK},\mathcal{A}}(\lambda)$ and refer to it as the advantage of $\mathcal{A}$ in the simulation-extractability game.

As noted, in our scheme, we will also require the extractor to be online (or straightline), which means that the extractor does not rewind the adversary.

4 Construction

We present our non-threshold batch encryption construction and prove its security in Section 4.1. We thresholdize the construction in Section 4.2. Both use an SE-NIZK which we present in Section 4.3. The SE-NIZK construction is a modified Schnorr Proof-of-Knowledge that captures the relation satisfied by valid ciphertexts given the encryption key.

4.1 Batch Encryption

As in the technical overview (Section 2), for any $i \in \mathbb{Z}_p$, let $p_i(X) \in \mathbb{F}[X]$ denote the linear fractions:

$$p_i(X) := \frac{1}{X+i}$$

The BE construction is presented in Construction 1, where we separate the SE-NIZK proof from the rest of the ciphertext for clarity. The SE-NIZK relation is

$$\mathcal{R}_{\mathsf{ek}[1]} := \left\{ (\mathsf{ct}, r) \in \left(\mathbb{G}_1^2 \times \mathbb{G}_T\right) \times \mathbb{Z}_p \;\middle|\; \mathsf{ct}[1] = [r]_1 \text{ and } \mathsf{ct}[2] = r \cdot \mathsf{ek}[1] \right\}$$

where ek is the encryption key output by $\mathsf{BE.Setup}$, ct is the ciphertext in $\mathsf{BE.Enc}$, and r is the randomness used. Note that the third ciphertext component $\mathsf{ct}[3] \in \mathbb{G}_T$, which carries the masked message, is part of the statement but is left unconstrained by the relation. This is essential for CCA security, as it ensures that mauling any component of a ciphertext (including the masked message) invalidates its proof. (See the proof of Theorem 1.)

Remark 1. In Construction 1, ciphertexts are verified by the pre-decryptor as part of the pre-decryption algorithm PreDec. We note that the ciphertexts are in fact publicly verifiable using the SE-NIZK and verifying them does not require possession of the secret key sk. Thus, one could separately define another algorithm Verify that simply executes $\mathsf{NIZK.Verify}$ and alternatively define a notion of "robust correctness" in which correct decryptions are obtained for batches that include verified ciphertexts. We omit this since it is auxiliary to the batch encryption primitive.

4.1.1 Correctness

By the completeness of the NIZK, $\mathsf{NIZK.Verify}(\mathsf{ct}_j, \pi_j) = 1$ for each $\mathsf{ct}_j \leftarrow \mathsf{Enc}(\mathsf{ek}, \mathsf{m}_j)$, $j \in [\ell]$.

Let r_j, $j \in [\ell]$, denote the randomness sampled for the ciphertext ct_j during encryption. Define $r_{-1} = r_0 := 0$. Then, expanding $L_{j,1}$, by Lemma 1,

$$\begin{aligned}
L_{j,1} &= e\left(r_j \cdot \left(\sum_{i=-1}^{\ell} [p_i(x)]_1 \right) - \sum_{i \in [\ell]} p_i(x)[r_i]_1 + \left(\sum_{\substack{i=-1 \\ i \neq j}}^{\ell} \frac{1}{j-i} \right) [r_j]_1 - \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{1}{j-i} [r_i]_1, [p_j(x)]_2 \right) \\
&\quad - \sum_{\substack{i \in [\ell] \\ i \neq j}} e\left(\frac{1}{j-i}([r_j]_1 - [r_i]_1), [p_i(x)]_2 \right) \\
&= \left[\left(\sum_{\substack{i=-1 \\ i \neq j}}^{\ell} (r_j - r_i) p_i(x) + \sum_{\substack{i=-1 \\ i \neq j}}^{\ell} \frac{r_j - r_i}{j-i} \right) p_j(x) \right]_T - \left[\sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{r_j - r_i}{j-i} \cdot p_i(x) \right]_T \\
&= \left[\sum_{\substack{i=-1 \\ i \neq j}}^{\ell} \frac{r_j - r_i}{j-i} p_i(x) - \sum_{\substack{i \in [\ell] \\ i \neq j}} \frac{r_j - r_i}{j-i} p_i(x) \right]_T
\end{aligned}$$

$\mathsf{Setup}(1^\lambda, 1^\ell)$:

- ▷ Sample $x \xleftarrow{\$} \mathbb{Z}_p$
- ▷ Output $\left(\mathsf{ek} \coloneqq \left(\sum_{i=-1}^{\ell} [p_i(x)]_1, [p_0(x)]_1\right), \mathsf{sk} \coloneqq x, \mathsf{dk} \coloneqq \left([p_i(x)]_2\right)_{i\in[\ell]}\right)$

$\mathsf{Enc}(\mathsf{ek}, m)$:

- ▷ Sample $r \xleftarrow{\$} \mathbb{Z}_p$
- ▷ Compute the ciphertext: $\mathsf{ct} \coloneqq ([r]_1, r \cdot \mathsf{ek}[1], r \cdot e(\mathsf{ek}[2], [1]_2) + m)$[a]
- ▷ Compute the SE-NIZK proof for $\mathcal{R}_{\mathsf{ek}[1]}$: $\pi \leftarrow \mathsf{NIZK.Prove}((\mathsf{ct}[1], \mathsf{ct}[2], \mathsf{ct}[3]), r)$
- ▷ Output (ct, π)

$\mathsf{PreDec}(\mathsf{sk}, (\mathsf{ct}_i, \pi_i)_{i\in[\ell]})$:

- ▷ Check the SE-NIZK proofs: $\mathsf{NIZK.Verify}((\mathsf{ct}_i[1], \mathsf{ct}_i[2], \mathsf{ct}_i[3]), \pi_i) \stackrel{?}{=} 1$ for all $i \in [\ell]$
- ▷ Output $\perp$ if any proof is invalid[b]
- ▷ Else output $\mathsf{sbk} \coloneqq \sum_{i\in[\ell]} p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1]$

$\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]})$:

- ▷ For each $j \in [\ell]$, compute, $\mathsf{tm}_j \coloneqq e\big(\mathsf{ct}_j[1],\ \mathsf{dk}_j\big)$
- ▷ For each $j \in [\ell]$,
  - Compute,

$$L_{j,1} \coloneqq e\left(\mathsf{ct}_j[2] - \mathsf{sbk} + \left(\sum_{\substack{i=-1\\ i\neq j}}^{\ell} \frac{1}{j-i}\right)\mathsf{ct}_j[1] - \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{ct}_i[1],\ \mathsf{dk}_j\right)$$
$$- e\left(\mathsf{ct}_j[1], \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{dk}_i\right) + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{tm}_i$$
$$L_{j,2} \coloneqq e\big(\mathsf{ct}_j[2],\ [1]_2\big) - e\left(\mathsf{ct}_j[1],\ \sum_{i\in[\ell]} \mathsf{dk}_i\right)$$

  - Solve for $r_j \cdot [p_0(x)]_T$ using $L_{j,1}$ and $L_{j,2}$ as $R \coloneqq j((j+1)L_{j,1} - L_{j,2})$.
  - Set $m_j \coloneqq \mathsf{ct}_j[3] - R$
- ▷ Output $(m_j)_{j\in[\ell]}$

[a] One can avoid the pairing during encryption by simply publishing $[p_0(x)]_T$ instead of $[p_0(x)]_1$ in ek.

[b] We can proceed to create a batch with only the ciphertexts with valid proofs and discard the invalid ones. However, the presentation of the decryption algorithm Dec becomes unwieldy and opaque due to indexing. Therefore, we omit this straightforward extension for brevity.

Construction 1: BE: Batch Encryption Construction Using Partial Fractions

$$= \left[\frac{r_j}{j+1}p_{-1}(x) + \frac{r_j}{j}p_0(x)\right]_T$$
$$= \frac{1}{j+1} \cdot [r_j p_{-1}(x)]_T + \frac{1}{j}[r_j p_0(x)]_T$$

And expanding $L_{j,2}$,

$$L_{j,2} = e\left(r_j \cdot \sum_{i=-1}^{\ell} [p_i(x)]_1, [1]_2\right) - e\left([r_j]_1, \sum_{i\in[\ell]} [p_i(x)]_2\right)$$
$$= [r_j p_{-1}(x)]_T + [r_j p_0(x)]_T$$

Using the linear independence of $L_{j,1}$ and $L_{j,2}$ per Lemma 1, we can solve for $[r_j \cdot p_0(x)]_T$:

$$\left[\frac{r_j}{x}\right]_T = j((j+1)L_{j,1} - L_{j,2})$$

Thus, each m_j is correctly decrypted.

4.1.2 Decryption Complexity

Both $L_{j,1}$ and $L_{j,2}$ require constant number of pairings to compute, if the decryptor has already obtained $\mathsf{tm}_j := e\big(\mathsf{ct}_j[1], \mathsf{dk}_j\big)$ for all $j \in [\ell]$. Thus, we can reduce the number of pairings per ciphertext from $O(\ell)$ to $O(1)$ by amortizing the computation of tm_j, $j \in [\ell]$, across the ℓ decryption operations. Then, the (amortized) number of pairings per ciphertext becomes 5.

As presented, decryptors must do $O(\ell)$ group operations per ciphertext, *i.e.*, the following 3 multi-scalar multiplications (MSMs) per ciphertext, to compute $L_{j,1}$, which results in a total of $O(\ell^2)$ group operations:

$$(1)\ \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{ct}_i[1] \qquad (2)\ \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{dk}_i \qquad (3)\ \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{tm}_i$$

We can reduce this quadratic complexity to quasi-linear by using roots of unity $\{\omega, \ldots, \omega^\ell\}$ instead of the integer indices $\{1, \ldots, \ell\}$ and Fast Fourier Transform (FFT) to compute the MSMs (1)-(2)-(3) collectively for all the ciphertexts. Here, we assume that ℓ is a power of two, and ω is a primitive (2ℓ)-th root of unity. In a nutshell, these MSMs can be interpreted as a cyclic convolution and thus, can be calculated for all $j \in \{\omega, \ldots, \omega^\ell\}$ with only $O(\ell \log \ell)$ group operation using FFTs.

4.1.3 Security

To prove security, we introduce a new static q-type assumption that is a static variant of the q-Decisional-Bilinear-Rank-Deficient-Diffie-Hellman assumption introduced in [24], which was shown to hold in the GBGM [27] in the same work. Our assumption is straightforwardly implied by theirs but for completeness, we provide a proof in Appendix A.1.

Assumption 1 (Static Decisional Bilinear Rank-Deficient Diffie-Hellman)**.** Let $x \xleftarrow{\$} \mathbb{Z}_p$ and $q \geq 1$, and let $\mathcal{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$ be a bilinear group. Define,

$$\mathcal{D}_R := \left\{ \left([r]_1,\ r \cdot \sum_{i=-1}^{q} \left[\frac{1}{x+i}\right]_1,\ [u]_T \right) \middle|\ r, u \xleftarrow{\$} \mathbb{Z}_p \right\}, \text{ and}$$

$$\mathcal{D}_t := \left\{ \left([r]_1, r \cdot \sum_{i=-1}^{q} \left[\frac{1}{x+i}\right]_1, r \cdot \left[\frac{1}{x}\right]_T \right) \middle| \; r \xleftarrow{\$} \mathbb{Z}_p \right\}$$

We state the q-Static-Decisional-Bilinear-Rank-Deficient-Diffie-Hellman (or q-StatRankDef for short) problem as follows: given

$$\mathsf{pp} := \left(\left[\frac{1}{x-1}\right]_1, \left[\frac{1}{x}\right]_1, \left[\frac{1}{x+1}\right]_1, \ldots, \left[\frac{1}{x+q}\right]_1, \left[\frac{1}{x+1}\right]_2, \ldots, \left[\frac{1}{x+q}\right]_2 \right),$$

and a sample ch, decide if $\mathsf{ch} \in \mathcal{D}_R$ or $\mathsf{ch} \in \mathcal{D}_t$.

Let,

$$\mathsf{Adv}^{\mathsf{StatRankDef}}_{q,\mathcal{A}} := \left| \Pr\left[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_R \right] - \Pr\left[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t \right] \right|$$

We say that the q-StatRankDef assumption holds in $\mathcal{G}$ if for every PPT algorithm $\mathcal{A}$, and for every polynomially bounded $q\colon \mathbb{Z} \to \mathbb{Z}$,

$$\mathsf{Adv}^{\mathsf{StatRankDef}}_{q(\lambda),\mathcal{A}(\lambda)} = \mathsf{negl}(\lambda)$$

We show that Construction 1 is B-IND-CCA-secure (Definition 3) using the simulation and straightline-extractability and zero-knowledge property of NIZK (*cf.* Section 3.5) and the hardness of q-StatRankDef.

Theorem 1. *The BE construction presented in Construction 1 is* B-IND-CCA*-secure assuming that* q-StatRankDef *holds and* NIZK *is complete, zero-knowledge and simulation-extractable.*

In the proof below, we assume that the adversary obtains,

$$\mathsf{ek} := \left([p_{-1}(x)]_1, [p_0(x)]_1, ([p_i(x)]_1)_{i\in[\ell]} \right)$$

as the encryption key from the challenger within the game $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell)$. This does not weaken the security analysis; since the adversary can calculate the encryption key used by the original scheme from the received values.

Proof. Suppose for contradiction BE is not B-IND-CCA-secure. Then, there exists $\ell \in \mathbb{N}$ and PPT adversary $\mathcal{A}$ that wins the game for batch encryption in Figure 2 with non-negligible probability, *i.e.*, $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A},\ell} > \mathsf{negl}(\lambda)$. We construct PPT adversary $\mathcal{B}$ such that $\mathsf{Adv}^{\mathsf{StatRankDef}}_{\ell,\mathcal{B}} > \mathsf{negl}(\lambda)$.

$\mathcal{B}$ receives as input $2\ell + 2$ group elements of the form

$$([p_{-1}(x)]_1, [p_0(x)]_1, [p_1(x)]_1, \ldots, [p_\ell(x)]_1, [p_1(x)]_2, \ldots, [p_\ell(x)]_2),$$

where $p_i(X) = \frac{1}{X+i}$, and a challenge $\mathsf{ch} = (\mathsf{ch}_1, \mathsf{ch}_2, \mathsf{ch}_3)$ from the ℓ-StatRankDef challenger. It sends $\mathsf{ek} := ([p_{-1}(x)]_1, [p_0(x)]_1, ([p_i(x)]_1)_{i\in[\ell]})$ and $\mathsf{dk} := ([p_i(x)]_2)_{i\in[\ell]}$ to $\mathcal{A}$. $\mathcal{B}$ also runs $(\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{td}_{\mathsf{Ext}}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$.

$\mathcal{B}$ responds to $\mathcal{A}$'s subsequent queries as follows,

- Pre-challenge queries: $\mathcal{A}$ sends a list $(\mathsf{ct}_i, \pi_i)_{i\in[\ell]}$. If $\mathsf{NIZK.Verify}(\mathsf{ct}_i, \pi_i) = 0$ for some $i \in [\ell]$, $\mathcal{B}$ responds with $\perp$, as PreDec would. Otherwise, $\mathcal{B}$ runs the (straightline) extractor Ext from the simulation extractability of NIZK (Definition 13) on the statement given by the entire ciphertext $\mathsf{ct}_i = (\mathsf{ct}_i[1], \mathsf{ct}_i[2], \mathsf{ct}_i[3])$ for each $i \in [\ell]$: $r_i \leftarrow \mathsf{Ext}(\mathsf{crs}, \mathsf{td}_{\mathsf{Ext}}, \mathsf{ct}_i, \pi_i)$. It responds to $\mathcal{A}$ with $\mathsf{sbk} := \sum_{i\in[\ell]} r_i \cdot [p_i(x)]_1$.

– Challenge round: $\mathcal{A}$ sends $(\mathsf{m}_0, \mathsf{m}_1)$. $\mathcal{B}$ samples bit $b \xleftarrow{\$} \{0,1\}$ and computes the ciphertext using the ℓ-StatRankDef challenge: $\mathsf{ct}^* := (\mathsf{ch}_1, \mathsf{ch}_2, \mathsf{ch}_3 + \mathsf{m}_b)$. $\mathcal{B}$ generates a simulated proof on the statement ct^* using the simulator of the NIZK: $\pi^* \leftarrow \mathsf{SimProve}(\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{ct}^*)$. It returns the challenge ciphertext (ct^*, π^*) to $\mathcal{A}$.
– Post-challenge queries: $\mathcal{A}$ sends a list $(\mathsf{ct}_i, \pi_i)_{i\in[\ell]}$. If the list contains the challenge ciphertext (ct^*, π^*), $\mathcal{B}$ responds with $\perp$, as the challenger in Figure 2 would. Otherwise, $\mathcal{B}$ responds as in the pre-challenge queries.

$\mathcal{A}$ outputs a bit b'. If $b = b'$, $\mathcal{B}$ outputs 1. Else it outputs 0 or 1 with equal probability.

Extraction step. We argue that, except with probability $\mathsf{negl}(\lambda)$, the pre-decryption key sbk that $\mathcal{B}$ computes from the extracted witnesses is *equal* to the output of $\mathsf{PreDec}(\mathsf{sk}, \cdot)$ on the same query. Every pair (ct_i, π_i) on which $\mathcal{B}$ runs Ext lies outside the set Q of statement–proof pairs output by $\mathsf{SimProve}$. Indeed, in the pre-challenge phase, $Q = \emptyset$, and in the post-challenge phase, $Q = \{(\mathsf{ct}^*, \pi^*)\}$ and all ciphertexts in batches that receive $\mathsf{sbk} \neq \perp$ differ from the challenge ciphertext. Hence, if with non-negligible probability ϵ some verifying queried pair yielded an invalid witness ($\mathsf{ct}_i[1] \neq [r_i]_1$ or $\mathsf{ct}_i[2] \neq r_i \cdot \mathsf{ek}[1]$), an adversary $\mathcal{A}_{\mathsf{SE}}$ that simulates $\mathcal{B}$'s game (sampling x and the ℓ-StatRankDef challenge itself, and obtaining π^* from its simulation oracle) and outputs a uniformly guessed queried pair would win the game of Definition 13 with non-negligible probability of at least $\epsilon/(\ell\, q(\lambda))$, where $q(\lambda)$ bounds the number of $\mathcal{A}$'s queries. This contradicts the simulation-extractability of NIZK. Therefore, except with negligible probability, all extracted witnesses are valid, $\mathsf{ct}_i[1] = [r_i]_1$ implies $p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1] = r_i \cdot [p_i(x)]_1$ for every $i \in [\ell]$, and thus $\mathsf{sbk} = \sum_{i\in[\ell]} r_i \cdot [p_i(x)]_1 = \sum_{i\in[\ell]} p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1]$, as claimed.

We next prove the claim $\mathsf{Adv}^{\mathsf{StatRankDef}}_{\ell,\mathcal{B}} \geq \frac{1}{2}\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A},\ell} - \mathsf{negl}(\lambda)$ by considering the following two cases:

ℓ-StatRankDef challenger samples from $\mathcal{D}_t$: By the zero-knowledge property of NIZK (Definition 12), the game $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A},\ell}$, where the proof π^* is generated by the algorithm Prove and the game, where the same proof is output by Sim, are indistinguishable for $\mathcal{A}$ except with probability $\mathsf{negl}(\lambda)$. Now, observe that in $\mathcal{B}$'s simulation, ek is identically distributed to the output of Setup for the BE. Moreover, by the extraction step above, $\mathcal{B}$'s responses to the pre- and post-challenge queries are identical to those of PreDec applied to them except with probability $\mathsf{negl}(\lambda)$. Finally, since the ℓ-StatRankDef challenger samples from $\mathcal{D}_t$, the challenge ciphertext ct^*, constructed by $\mathcal{B}$, is identically distributed to that of Enc. Therefore, $\mathcal{A}$'s transcript in $\mathcal{B}$'s simulation is indistinguishable from that of $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}$ in Figure 2 except with probability $\mathsf{negl}(\lambda)$.

ℓ-StatRankDef challenger samples from $\mathcal{D}_R$: By the same arguments above, $\mathcal{A}$'s transcript in $\mathcal{B}$'s simulation is indistinguishable from that of $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}$ in Figure 2 except with probability $\mathsf{negl}(\lambda)$. Moreover, since the ℓ-StatRankDef challenger samples from $\mathcal{D}_R$, the ciphertext ct^* perfectly hides m_b as it is independent of b. Thus, $\mathcal{A}$ outputs $b' = b$ with probability $\frac{1}{2}$; since b is chosen uniformly at random. Then, the probability that $\mathcal{B}$ outputs 1 is, $\Pr[b = b'] \cdot \Pr[\mathcal{B} \text{ outputs } 1 | b = b'] + \Pr[b \neq b'] \cdot \Pr[\mathcal{B} \text{ outputs } 1 | b \neq b'] = \frac{3}{4}$.

So $\mathsf{Adv}^{\mathsf{StatRankDef}}_{\ell,\mathcal{B}}$ is,

$$\left|\Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_R\right] - \Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t\right]\right|$$
$$\geq \left|\frac{3}{4} - \left(\Pr\left[b = b'\right] \cdot \Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t, b = b'\right] +\right.\right.$$

$$
\begin{aligned}
&\Pr\big[b \neq b'\big] \cdot \Pr\Big[\mathcal{B}(\mathsf{pp}, \mathsf{ch}) = 1 \;\Big|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t, b \neq b'\Big]\Big) \Big| - \mathsf{negl}(\lambda) \\
\geq \Big| &\frac{3}{4} - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell) = 1] \cdot 1 \\
&- (1 - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell) = 1]) \cdot \frac{1}{2} \Big| - \mathsf{negl}(\lambda) \\
= \frac{1}{2} \Big| &\frac{1}{2} - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell) = 1] \Big| - \mathsf{negl}(\lambda) \\
= \frac{1}{2} &\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}} - \mathsf{negl}(\lambda)
\end{aligned}
$$

Therefore, $\mathsf{Adv}^{\mathsf{StatRankDef}}_{\ell,\mathcal{B}} > \mathsf{negl}(\lambda)$, a contradiction. □

4.2 Batch Threshold Encryption

Let $(\lambda_j)_{j\in[N]} \in (\mathbb{Z}_p)^N$ denote the Lagrange coefficients for t-of-N Shamir secret sharing, *i.e.*, for any set of secret shares $(x_j)_{j\in[N]}$ for some $x \in \mathbb{Z}_p$ and any subset $T \subseteq [N]$ of size $|T| = t$, it holds that $\sum_{j\in T} \lambda_j x_j = x$.

The protocol is presented in Construction 2, where we again separate the SE-NIZK proof from the rest of the ciphertext for clarity.

Each pre-decryption key share sbk_i is a single group element, and the combiner algorithm does not require any interaction among the parties, making the communication complexity $O(N)$. Moreover, the scheme inherits the same amortized $O(1)$ complexity for the pairing operations needed for decryption. However, the secret key of each party consists of ℓ field elements.

Remark 2. It is possible to design a batch threshold encryption scheme, where each secret key is a single field element, namely, a t-of-N secret share of x. To achieve this, we can use the two-round protocol by Wang et al. [31] originally designed for thresholdizing the Boneh-Boyen signature scheme [9]. In the protocol, the parties first jointly compute and publish $a_i = \sigma_i(x + i) \in \mathbb{Z}_p$ in the clear for $i \in [\ell]$, where each $\sigma_i \leftarrow \mathbb{Z}_p$, $i \in [\ell]$, is also t-of-N secret shared among them. Then, each party j publishes $\mathsf{sbk}_j = \sum_{i\in[\ell]}(\sigma^i_j/a_i)\mathsf{ct}_i[1]$, where σ^i_j is party j's share of σ_i. Finally, given a subset T of t honest parties, the pre-decryption key is computed as $\sum_{j\in T} \lambda_j \mathsf{sbk}_j = \sum_{i\in[\ell]} \frac{1}{x+i}\mathsf{ct}_i[1]$, as required.

$\mathsf{Setup}(1^\lambda, 1^\ell, 1^N, 1^t)$:

- ▷ Sample $x \xleftarrow{\$} \mathbb{Z}_p$
- ▷ For $i \in [\ell]$, sample t-of-N Shamir secret shares $\left(s_j^i\right)_{j\in[N]} \in (\mathbb{Z}_p)^N$ of $p_i(x)$.
- ▷ Output, $\begin{pmatrix} \mathsf{ek} & \coloneqq \left(\sum_{i=-1}^{\ell} [p_i(x)]_1, [p_0(x)]_1\right), \\ (\mathsf{sk}_j & \coloneqq (s_j^i)_{i\in[\ell]})_{j\in[N]}, \\ \mathsf{dk} & \coloneqq ([p_i(x)]_2)_{i\in[\ell]} \end{pmatrix}$

$\mathsf{Enc}(\mathsf{pk}, m)$: same as in Construction 1.

$\mathsf{PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i, \pi_i)_{i\in[\ell]})$:

- ▷ Check the NIZK proofs: $\mathsf{NIZK.Verify}(\mathsf{ct}_i, \pi_i) \stackrel{?}{=} 1$ for all $i \in [\ell]$
- ▷ Output $\perp$ if any proof is invalid
- ▷ Else parse $\mathsf{sk}_j = (s_j^i)_{i\in[\ell]}$ and output $\mathsf{sbk}_j \coloneqq \sum_{i\in[\ell]} s_j^i \cdot \mathsf{ct}_i[1]$

$\mathsf{Combine}(T \subseteq [N], (\mathsf{sbk}_j)_{j\in T}) \to \mathsf{sbk}$:

- ▷ Output $\mathsf{sbk} \coloneqq \sum_{j\in T} \lambda_j \cdot \mathsf{sbk}_j$

$\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]})$: same as in Construction 1.

Construction 2: Batch Threshold Encryption construction BTE

4.2.1 Correctness

Let $(\mathsf{ct}_i)_{i\in[\ell]}$ denote ℓ correctly generated ciphertexts. Then, for any t correctly generated pre-decryption key shares $(\mathsf{sbk}_j)_{j\in T}$, $T \subseteq [N]$, for these ciphertexts, it holds that

$$\sum_{j\in T} \lambda_j \cdot \mathsf{sbk}_j = \sum_{j\in T} \lambda_j \sum_{i\in[\ell]} s_j^i \cdot \mathsf{ct}_i[1] = \sum_{i\in[\ell]} \left(\sum_{j\in T} \lambda_j s_j^i\right) \cdot \mathsf{ct}_i[1] = \sum_{i\in[\ell]} p_i(x) \cdot \mathsf{ct}_i[1]$$

In other words, output of the combiner algorithm is equal to the correctly generated pre-decryption key of the BE scheme in Construction 1. Then, as the decryption algorithms are identical, correctness follows from the correctness of the BE scheme.

4.2.2 Security

Theorem 2. BTE *presented in Construction 2 is* B-IND-CCA*-secure assuming* BE *(Construction 1) is* B-IND-CCA*-secure.*

Proof. We prove the security of BTE by reducing it to the security of BE. Namely, for any $\ell, N, t \in \mathbb{N}$, we show that for every PPT adversary $\mathcal{A}$ that attacks the game $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^t)$ in Figure 2, there exists a PPT adversary $\mathcal{B}$ such that $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A},\ell,N,t} = \mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{B},\ell}$. The adversary $\mathcal{B}$ plays the role of challenger to $\mathcal{A}$ as described below.

We again assume that the adversary $\mathcal{B}$ receives,

$$\mathsf{ek} \coloneqq \left([p_{-1}(x)]_1, [p_0(x)]_1, ([p_i(x)]_1)_{i\in[\ell]}\right)$$

and dk from the challenger within the game $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell)$. It forwards them to $\mathcal{A}$, and upon receiving some subset $S \subseteq [N]$ of size $t-1$, does the following for each $i \in [\ell]$:

- For $j' \in S$, $\mathcal{B}$ samples $s^i_{j'}$ uniformly at random: $s^i_{j'} \xleftarrow{\$} \mathbb{Z}_p$.
- It then finds the Lagrange coefficients $\left(\lambda_{j',j}\right)_{j'\in S\cup\{0\}, j\in[N]\setminus S}$ for the unique, degree-$(t-1)$ polynomial $f_i(X)$ such that $f_i(0) = p_i(x)$ and $f_i(j') = s^i_{j'}$ for $j' \in S$: $f_i(j) = \lambda_{0,j} f_i(0) + \sum_{j'\in S} \lambda_{j',j} f_i(j')$ for $j \in [N] \setminus S$.
- For $j \in [N] \setminus S$, $\mathcal{B}$ computes $\left[s^i_j\right]_1$ via interpolation in the exponent:

$$\left[s^i_j\right]_1 \coloneqq \lambda_{0,j} \cdot [p_i(x)]_1 + \sum_{j'\in S} \lambda_{j',j} \cdot \left[s^i_{j'}\right]_1$$

Finally, $\mathcal{B}$ returns $\mathsf{sk}_{j'} = \left(s^i_{j'}\right)_{i\in[\ell]}$, $j' \in S$, to $\mathcal{A}$.

Next, $\mathcal{B}$ responds to $\mathcal{A}$'s pre-challenge queries as follows: When $\mathcal{A}$ sends a list $(\mathsf{ct}_i, \pi_i)_{i\in[\ell]}$, $\mathcal{B}$ forwards it to its challenger within the game $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{A}}(1^\lambda, 1^\ell)$. Upon receiving sbk, it checks if $\mathsf{sbk} = \bot$. If so, it sets all pre-decryption key shares to $\bot$. Otherwise, for $j' \in S$, it computes $\mathsf{sbk}_{j'}$ using $\left(s^i_{j'}\right)_{i\in[\ell], j'\in S}$:

$$\mathsf{sbk}_{j'} = \sum_{i\in[\ell]} s^i_{j'} \cdot \mathsf{ct}_i[1]$$

For $j \in [N] \setminus S$, it computes sbk_j via interpolation in the exponent:

$$\mathsf{sbk}_j \coloneqq \lambda_{0,j} \cdot \mathsf{sbk} + \sum_{j'\in S} \lambda_{j',j} \sum_{i\in[\ell]} s^i_{j'} \cdot \mathsf{ct}_i[1]$$

It returns sbk_j, $j \in [N]$.

In the challenge round, upon receiving $(\mathsf{m}_0, \mathsf{m}_1)$ from $\mathcal{A}$, $\mathcal{B}$ forwards it to its own challenger within the game $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{B}}(1^\lambda, 1^\ell)$ and relays the returned challenge ciphertext ct_b to $\mathcal{A}$. For the post-challenge queries, if the list sent by $\mathcal{A}$ contains the challenge ciphertext ct_b, $\mathcal{B}$ responds with $\bot$ without forwarding the query, exactly as the BTE challenger in Figure 2 would; note that Enc, and hence the distribution of the challenge ciphertext, is identical in the two games. Otherwise, $\mathcal{B}$ responds as in the pre-challenge queries. If $\mathcal{A}$ outputs a bit b, $\mathcal{B}$ outputs the same bit b.

Note that $\mathcal{A}$'s transcript is identically distributed to $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A},\ell,N,t}$ in Figure 2. Therefore, $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A},\ell,N,t} = \mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE},\mathcal{B},\ell}$, which completes the proof. □

4.2.3 Robustness

To ensure robustness of our BTE scheme, we can introduce per-party verification keys to Construction 2:

Theorem 3. $\mathsf{BTE}_{\mathrm{robust}}$ *presented in Construction 3 is* $\mathsf{B\text{-}IND\text{-}CCA}$*-secure assuming the* $\mathsf{B\text{-}IND\text{-}CCA}$*-security of* BE*. Moreover, it satisfies accurate blaming and consistency.*

$\mathsf{Setup}(1^\lambda, 1^\ell, 1^N, 1^t)$:

- ▷ Sample $x \xleftarrow{\$} \mathbb{Z}_p$
- ▷ For $i \in [\ell]$, sample t-of-N Shamir secret shares $\left(s_j^i\right)_{j\in[N]} \in (\mathbb{Z}_p)^N$ of $p_i(x)$.
- ▷ Let $\mathsf{vk}_j^i \coloneqq \left[s_j^i\right]_2$.
- ▷ Output

$$\left(\mathsf{ek}, \left(\mathsf{sk}_j \coloneqq (s_j^i)_{i\in[\ell]}\right)_{j\in[N]}, \left(\mathsf{vk}_j \coloneqq (\mathsf{vk}_j^i)_{i\in[\ell]}\right)_{j\in[N]}, \mathsf{dk}\right)$$

$\mathsf{Combine}(\mathsf{vk}, T \subseteq [N], (\mathsf{sbk}_j)_{j\in T}) \to \mathsf{sbk}$:

- ▷ For each $j \in T$, check if $\sum_{i\in[\ell]} e(\mathsf{ct}_i[1], \mathsf{vk}_j^i) \stackrel{?}{=} e(\mathsf{sbk}_j, [1]_2)$. If not, blame j.
- ▷ If no party is blamed, output $\mathsf{sbk} \coloneqq \sum_{j\in T} \lambda_j \cdot \mathsf{sbk}_j$

Construction 3: Robust Batch Threshold Encryption construction $\mathsf{BTE}_{\mathrm{robust}}$. The algorithms $\mathsf{Enc}(\mathsf{pk}, m)$, $\mathsf{PreDec}(\mathsf{sk}_j, (\mathsf{ct}_i, \pi_i)_{i\in[\ell]})$, and $\mathsf{Dec}(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]})$ are the same as in Construction 2.

Proof. Proof of B-IND-CCA-security is similar to that of Theorem 2, except that $\mathcal{B}$ now has to return a verification key vk to $\mathcal{A}$. For this purpose, it sets $\mathsf{vk}_j^i \coloneqq \left[s_j^i\right]_2$ for $j \in S$, and finds vk_j^i for $j \in [N] \setminus S$ by interpolating in the exponent. Thus, $\mathcal{B}$ ensures that $\mathcal{A}$'s transcript is identically distributed to $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BTE},\mathcal{A},\ell,N,t}$ in Figure 2, now extended with verification keys.

For accurate blaming, note that a correctly generated pre-decryption key share will never lead to blame. For consistency, observe that any valid collection of pre-decryption key shares gives the same pre-decryption key that would have been output by the BE scheme (Construction 1), which would in turn decrypt the ciphertexts to the same message by correctness. □

4.3 NIZK for Ciphertext Relation

In Construction 4, we present our SE-NIZK design, NIZK, used by our batch encryption schemes. The construction is essentially a Schnorr non-interactive proof of knowledge [23] with an additional constraint. A similar construction is used in [14] but our proof is smaller by one group (and one field) element due to the shorter witness. We prove that NIZK is complete, sound, zero-knowledge and simulation-extractable in Lemmas 2 to 5.

Our SE-NIZK is defined for a family of relations indexed by a group element $x_0 \in \mathbb{G}_1$. Let $\mathcal{X} \coloneqq \mathbb{G}_1^2 \times \mathbb{G}_T$, $\mathcal{W} \coloneqq \mathbb{Z}_p$ and $x_0 \in \mathbb{G}_1$. Define

$$\mathcal{R}_{x_0} \coloneqq \{(x = (x_1, x_2, x_3), w) \in \mathcal{X} \times \mathcal{W} \mid x_1 = [w]_1,\ x_2 = w \cdot x_0\}\,.$$

The above relation corresponds exactly to the public parameters and ciphertext in our BE constructions: x_0 corresponds to $\mathsf{ek}[1]$ generated as part of the setup algorithm and is fixed, while x_1, x_2 and x_3 correspond to the three components of the ciphertext, and w corresponds to the randomness used during encryption. (See Constructions 1 and 2.) The component x_3 is a tag that the relation leaves unconstrained; including it in the statement, and thus in the Fiat–Shamir hash below, binds the proof to the entire ciphertext.

$\mathsf{Setup}(1^\lambda)$:

▷ Output $\perp$

$\mathsf{Prove}(x, w)$:

▷ Parse $x =: (x_1, x_2, x_3)$
▷ Sample $w' \xleftarrow{\$} \mathcal{W}$ and let $x'_1 := [w']_1$ and $x'_2 := w' \cdot x_0$
▷ Let $c \leftarrow H(x_0, x_1, x_2, x_3, x'_1, x'_2)$
▷ Let $\hat{w} := w' + cw$
▷ Output $\pi := (\hat{w}, x'_1, x'_2)$

$\mathsf{Verify}(x, \pi)$:

▷ Parse $x =: (x_1, x_2, x_3)$ and $\pi =: (\hat{w}, x'_1, x'_2)$
▷ Compute $c \leftarrow H(x_0, x_1, x_2, x_3, x'_1, x'_2)$
▷ Output $(x'_1 + c \cdot x_1 \stackrel{?}{=} [\hat{w}]_1) \wedge (x'_2 + c \cdot x_2 \stackrel{?}{=} \hat{w} \cdot x_0)$

Construction 4: NIZK: SE-NIZK construction for $\mathcal{R}_{x_0}$

Lemma 2. NIZK *is complete (Definition 10).*

Proof. Let $(x = (x_1, x_2, x_3), w) \in \mathcal{R}_{x_0}$. Then for c as defined in Construction 4, $x'_1 + c \cdot x_1$, is equal to $[w']_1 + c[w]_1 = [w' + cw]_1 = [\hat{w}]_1$. And $x'_2 + c \cdot x_2$ equals $w' \cdot x_0 + c \cdot w \cdot x_0 = (w' + cw) \cdot x_0 = \hat{w} \cdot x_0$. Therefore, $\mathsf{Verify}(x, \pi = (\hat{w}, x'_1, x'_2)) = 1$. We conclude that NIZK satisfies completeness. □

Lemma 3. NIZK *is sound (Definition 11).*

Proof. Suppose $\mathsf{Verify}(x = (x_1, x_2, x_3), \pi = (\hat{w}, x'_1, x'_2)) = 1$ for some $x \in \mathcal{X}$ and $\pi \in \mathbb{Z}_p \times \mathbb{G}_1 \times \mathbb{G}_1$. We can assume that $x_i = [a_i]_1$ for $i = 0, 1, 2$, $x'_1 = [b_1]_1$ and $x'_2 = b_2 \cdot x_0 = b_2 \cdot [a_0]_1$ for some $a_0, a_1, a_2, b_1, b_2 \in \mathbb{Z}_p$.

Then, $\mathsf{Verify}(x, \pi) = 1$ implies $c \cdot [a_1]_1 + [b_1]_1 = [\hat{w}]_1$ and $c \cdot [a_2]_1 + b_2 \cdot [a_0]_1 = \hat{w} \cdot [a_0]_1$ for $c = H(x_0, x_1, x_2, x_3, x'_1, x'_2)$. Or equivalently, $ca_1 + b_1 = \hat{w}$ and $ca_2 + b_2 a_0 = \hat{w} a_0$. Substituting $\hat{w}$, we obtain

$$ca_2 + b_2 a_0 = (ca_1 + b_1)a_0 \implies c(a_2 - a_1 a_0) + a_0(b_2 - b_1) = 0\,.$$

If $(a_2 - a_1 a_0) \neq 0$, then $c = \frac{-a_0(b_2 - b_1)}{(a_2 - a_1 a_0)}$. Since c is the output of a Random Oracle on input determined by a_0, a_1, a_2, b_1, b_2 and x_3, P^* satisfies the above relation with negligible probability.

Therefore with overwhelming probability, $a_2 - a_1 a_0 = 0$ which implies $a_2 = a_1 a_0$. By definition $x_i = [a_i]_1$, and so we have $x_1 = [a_1]_1$ and $x_2 = a_1 \cdot x_0$. Thus, $(x, a_1) \in \mathcal{R}_{x_0}$.

□

Lemma 4. NIZK *is zero-knowledge (Definition 12).*

Proof. We construct a simulator Sim in the Random Oracle Model by programming the Random Oracle. Let $(x, w) \in \mathcal{R}_{x_0}$. Sim takes as input the statement $x = (x_1, x_2, x_3)$ and proceeds as follows:

– Sample $\hat{w}, c \xleftarrow{\$} \mathbb{Z}_p$
– Compute $x'_1 = [\hat{w}]_1 - cx_1$, and $x'_2 = \hat{w} \cdot x_0 - cx_2$
– Set $H(x_0, x_1, x_2, x_3, x'_1, x'_2) \leftarrow c$
– Output $\pi := (\hat{w}, x'_1, x'_2)$

It follows immediately that $(x, \mathsf{Sim}(x))$ is identically distributed to $(x, \mathsf{NIZK.Prove}(x, w))$. Thus, NIZK is zero-knowledge. □

To prove simulation-extractability, we assume x_0 is drawn from a particular distribution. This distribution is exactly the distribution of $\mathsf{ek}[1]$ for encryption key ek output by the setup algorithm BE.Setup in Constructions 1 and 2. We also assume q-StatRankDef holds (Assumption 1), which was used to prove the security of BE in Section 4.1.3. Let $m \geq 1$ be a fixed parameter.

Lemma 5. NIZK *is straightline simulation-extractable (Definition 13) assuming Assumption 1 holds and* $x_0 = \sum_{i=-1}^{m} \left[\frac{1}{x+i}\right]$ *for* $x \xleftarrow{\$} \mathbb{Z}_p$.

Proof. We prove the straightline simulation-extractability of NIZK in the AGM and ROM using techniques from [21]. Let SimProve be Sim from the simulator for zero-knowledge as defined in the proof of Lemma 4. SimSetup outputs $(\perp, \perp, \perp)$. Thus, we omit $\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{td}_{\mathsf{Ext}}$ from here on. At the beginning, $\mathcal{A}$ knows the description of the NIZK which includes the following $\mathbb{G}_1$ elements: (i) x_0, and (ii) the generator, $[1]_1$. Thus, for its first query to $\mathcal{O}_{\mathsf{Sim}}$, it constructs statement $x^{(1)} = \left(x_1^{(1)}, x_2^{(1)}, x_3^{(1)}\right) \in \mathcal{X} = \mathbb{G}_1^2 \times \mathbb{G}_T$, whose first two components can be written as

$$x_1^{(1)} = a_1^{(1)} \cdot x_0 + a_2^{(1)} \cdot [1]_1, \text{ and } x_2^{(1)} = b_1^{(1)} \cdot x_0 + b_2^{(1)} \cdot [1]_1$$

for some $a_1^{(1)}, a_2^{(1)}, b_1^{(1)}, b_2^{(1)} \in \mathbb{Z}_p$, and Ext is also able to see these coefficients since $\mathcal{A}$ is algebraic. The tag component $x_3^{(1)} \in \mathbb{G}_T$ enters only the random-oracle input and plays no role in the algebraic equations below.

SimProve responds with a simulated proof $\pi^{(1)} = \left(\hat{w}^{(1)}, x_1^{(1')}, x_2^{(1')}\right)$ such that it satisfies

$$x_1^{(1')} = \hat{w}^{(1)} \cdot [1]_1 - c^{(1)} \cdot x_1^{(1)}, \text{ and } x_2^{(1')} = \hat{w}^{(1)} \cdot x_0 - c^{(1)} \cdot x_2^{(1)},$$

by definition of Sim. Here, $c^{(1)} \xleftarrow{\$} \mathbb{Z}_p$ and is equal to $H(x_0, x_1^{(1)}, x_2^{(1)}, x_3^{(1)}, x_1^{(1')}, x_2^{(1')})$ by programming the Random Oracle. Note that given the scalars $\hat{w}^{(1)}$ and $c^{(1)}$, $\mathcal{A}$ can compute $x_1^{(1')}$ and $x_2^{(1')}$ itself. Therefore, in the view of Ext, any $\mathbb{G}_1$ element $\mathcal{A}$ can compute after calling $\mathcal{O}_{\mathsf{Sim}}$ is still a linear combination of x_0 and $[1]_1$. This holds for every query $\mathcal{A}$ makes to $\mathcal{O}_{\mathsf{Sim}}$ as well as the final statement-proof pair it outputs.

After making sufficiently many queries to $\mathcal{O}_{\mathsf{Sim}}$, suppose $\mathcal{A}$ eventually outputs $x^* = (x_1^*, x_2^*, x_3^*)$ and $\pi^* = \left(\hat{w}^*, x_1^{*'}, x_2^{*'}\right)$ such that $\mathsf{Verify}(x^*, \pi^*) = 1$ and $(x^*, \pi^*) \notin Q$.

We first argue that the challenge $c^* = H(x_0, x_1^*, x_2^*, x_3^*, x_1^{*'}, x_2^{*'})$ used to verify π^* is a *fresh* random-oracle value, *i.e.*, it was not programmed by SimProve. Suppose instead that it was programmed during some query to $\mathcal{O}_{\mathsf{Sim}}$. Since the programmed point determines the entire hash input, that query was on the statement x^* itself, and the simulated proof $\pi = (\hat{w}, x_1^{*'}, x_2^{*'})$ returned by $\mathcal{O}_{\mathsf{Sim}}$ shares the commitments of π^*. The first verification equation determines the response uniquely given

the statement, the commitments, and the challenge: $[\hat{w}^*]_1 = x_1^{*'} + c^* \cdot x_1^*$; the simulated response $\hat{w}$ satisfies the same equation by construction of Sim. Hence $\hat{w}^* = \hat{w}$, so $\pi^* = \pi$ and $(x^*, \pi^*) \in Q$, contradicting the winning condition of Definition 13. Therefore, c^* was sampled by the random oracle *after* the group elements $x_1^*, x_2^*, x_3^*, x_1^{*'}, x_2^{*'}$ and as $\mathcal{A}$ is algebraic, after the coefficients of their representations in terms of x_0 and $[1]_1$ were determined.[8] This freshness argument is used below whenever we argue that c^* is sampled after certain coefficients are determined.

Then, Ext knows scalars $a_1^*, a_2^*, b_1^*, b_2^*, a_1^{*'}, a_2^{*'}, b_1^{*'}, b_2^{*'} \in \mathbb{Z}_p$ such that

$$x_1^* = a_1^* \cdot x_0 + a_2^* \cdot [1]_1, \text{ and } x_2^* = b_1^* \cdot x_0 + b_2^* \cdot [1]_1 \tag{5}$$

$$x_1^{*'} = a_1^{*'} \cdot x_0 + a_2^{*'} \cdot [1]_1, \text{ and } x_2^{*'} = b_1^{*'} \cdot x_0 + b_2^{*'} \cdot [1]_1 \,. \tag{6}$$

Moreover, x^* and π^* must satisfy the following equations by definition of Verify:

$$\begin{aligned} x_1^{*'} - \hat{w}^* \cdot [1]_1 + c^* \cdot x_1^* &= 0 \\ x_2^{*'} - \hat{w}^* \cdot x_0 + c^* \cdot x_2^* &= 0, \end{aligned} \tag{7}$$

Substituting Equations (5) and (6) in Equation (7), we obtain

$$\begin{aligned} a_1^{*'} \cdot x_0 + a_2^{*'} \cdot [1]_1 - \hat{w}^* \cdot [1]_1 + c^* \cdot (a_1^* \cdot x_0 + a_2^* \cdot [1]_1) &= 0 \\ b_1^{*'} \cdot x_0 + b_2^{*'} \cdot [1]_1 - \hat{w}^* \cdot x_0 \; + c^* \cdot (b_1^* \cdot x_0 + b_2^* \cdot [1]_1) &= 0, \end{aligned}$$

which we can rearrange as

$$\left(a_1^{*'} + c^* a_1^*\right) \cdot x_0 + \left(a_2^{*'} - \hat{w}^* + c^* a_2^*\right) \cdot [1]_1 = 0 \tag{8}$$

$$\left(b_1^{*'} - \hat{w}^* + c^* b_1^*\right) \cdot x_0 + \left(b_2^{*'} + c^* b_2^*\right) \qquad \cdot [1]_1 = 0 \,. \tag{9}$$

Suppose $a_1^{*'} + c^* a_1^* \neq 0$. Then, Ext can compute the discrete logarithm of x_0 as $a_0 := \frac{-\left(a_2^{*'} - \hat{w}^* + c^* a_2^*\right)}{a_1^{*'} + c^* a_1^*}$. Therefore, we have $a_0 = \sum_{i=-1}^{m} \frac{1}{x+i}$ for uniformly random x by assumption on x_0. By clearing the denominator and rearranging, we obtain a degree-$(m+2)$ polynomial relation over x. We can solve for $x \in \mathbb{F}$ using efficient (polynomial in m) polynomial factorization algorithms over $\mathbb{F}$ such as Berlekamp's algorithm [8,28]. A solution to x can be used to easily break Assumption 1 for $q = m$ by computing $[p_0(x)]_2$ and checking if $\mathsf{ch}_3 \stackrel{?}{=} e(\mathsf{ch}_1, [p_0(x)]_2)$.[9] Since we assume q-$\mathsf{StatRankDef}$ (Assumption 1) holds, $a_1^{*'} + c^* a_1^* = 0$ with overwhelming probability. This implies two equations: first, since c^* is sampled *after* $a_1^{*'}$ and a_1^* are determined by $\mathcal{A}$, we must have $a_1^{*'} = a_1^* = 0$ except with negligible probability; second, this implies $\left(a_2^{*'} - \hat{w}^* + c^* a_2^*\right) = 0$ and so we have, $\hat{w}^* = \left(a_2^{*'} + c^* a_2^*\right)$ or equivalently, $a_2^* = \frac{\hat{w}^* - a_2^{*'}}{c^*}$.

[8] We ignore the standard failure events of random-oracle programming, namely that $\mathsf{SimProve}$ programs a point that was previously queried, or that two programmed points collide; since the commitments output by Sim are uniformly distributed, these events occur with probability $O(q_H^2/p)$, where q_H denotes the total number of random-oracle queries.

[9] More formally, we reduce the security of this case to m-$\mathsf{StatRankDef}$ where we construct x_0 using pp as defined in Assumption 1.

Applying the analysis of Equation (8) to Equation (9), we also obtain $\left(b_1^{*'} - \hat{w}^* + c^* b_1^*\right) = 0$ with overwhelming probability. This implies $\hat{w}^* = \left(b_1^{*'} + c^* b_1^*\right)$ or equivalently, $b_1^* = \frac{\hat{w}^* - b_1^{*'}}{c^*}$; and $b_2^{*'} + c^* b_2^* = 0$ and so, $b_2^{*'} = b_2^* = 0$ since c^* was sampled after $b_2^{*'}$ and b_2^* were determined by $\mathcal{A}$.

Therefore, $\hat{w}^* = \left(a_2^{*'} + c^* a_2^*\right) = \left(b_1^{*'} + c^* b_1^*\right)$, which implies $a_2^{*'} = b_1^{*'}$ with overwhelming probability, as c^* was sampled after $a_2^{*'}, a_2^*, b_1^{*'}, b_1^*$. Recall that we wrote $x_1^* = a_1^* \cdot x_0 + a_2^* \cdot [1]_1$. Plug in the values above to obtain

$$x_1^* = 0 \cdot x_0 + \frac{\hat{w}^* - a_2^{*'}}{c^*} \cdot [1]_1 = \frac{\hat{w}^* - a_2^{*'}}{c^*} \cdot [1]_1 \,.$$

Similarly, we wrote $x_2^* = b_1^* \cdot x_0 + b_2^* \cdot [1]_1$. Plug in the values above to obtain

$$x_2^* = \frac{\hat{w}^* - b_1^{*'}}{c^*} \cdot x_0 + 0 \cdot [1]_1 = \frac{\hat{w}^* - b_1^{*'}}{c^*} \cdot x_0 \,.$$

Let $w^* = \frac{\hat{w}^* - a_2^{*'}}{c^*} = \frac{\hat{w}^* - b_1^{*'}}{c^*}$ since $a_2^{*'} = b_1^{*'}$.

By definition of $\mathcal{R}_{x_0}$, we want to extract w such that $x_1^* = w \cdot [1]_1$, *and* $x_2^* = w \cdot x_0$. Indeed, w^* satisfies these constraints and thus, we successfully extract the witness as w^* in a straightline manner with overwhelming probability.

□

5 Batch Encryption With Shorter Ciphertexts

In this section, we formally construct the batch encryption scheme with a ciphertext size of only $|\mathbb{G}_1| + |\mathbb{G}_T|$. For simplicity, we focus on the non-threshold variant. It can be tresholdized identically to Construction 2.

Re-introducing notation from the overview (Section 2.7), let

$$w_j(X) := q_j(X) + \frac{1}{j+1} p_{-1}(X) + \frac{1}{j} p_0(X) \,.$$

The BE construction with shorter ciphertexts, $\mathsf{BE}_{\mathrm{short}}$, is presented in Construction 5. The required SE-NIZK is for the discrete logarithm relation, which is much simpler than the SE-NIZK for $\mathcal{R}_{\mathsf{ek}[1]}$ in Section 4.1, and is concretely more efficient: The proof is only $|\mathbb{G}_1| + |\mathbb{Z}_p|$ bits. For completeness, the discrete logarithm relation is

$$\mathcal{R}^{\mathsf{DLOG}} := \{((h, y), x) \in (\mathbb{G}_1 \times \mathbb{G}_T) \times \mathbb{Z}_p \mid h = [x]_1\} \,.$$

As in Section 4.1, the statement carries the second ciphertext component $y = \mathsf{ct}[2] \in \mathbb{G}_T$, which holds the masked message, as a tag that is left unconstrained by the relation: it enters the proof only through the Fiat–Shamir hash. This binds the proof to the entire ciphertext, which is essential for CCA security. A Schnorr non-interactive proof of knowledge for $\mathcal{R}^{\mathsf{DLOG}}$ can be shown to be simulation-extractable in AGM [1,21]; the tag enters only the hash, and the analysis is analogous to that of Lemma 5. We denote the SE-NIZK by $\mathsf{NIZK}^{\mathsf{DLOG}}$ and describe it for completeness in Construction 6.

$\mathsf{Setup}(1^\lambda, 1^\ell)$:

- ▷ Sample $x \xleftarrow{\$} \mathbb{Z}_p$
- ▷ Output $\Big(\mathsf{ek} \coloneqq [p_0(x)]_1, \mathsf{sk} \coloneqq x, \mathsf{dk} \coloneqq \Big(\big([p_i(x)]_2\big)_{i\in[\ell]}, \big([w_i(x)]_2\big)_{i\in[\ell]}, [p_{-1}(x)+p_0(x)]_2\Big)\Big)$

$\mathsf{Enc}(\mathsf{ek}, m)$:

- ▷ Sample $r \xleftarrow{\$} \mathbb{Z}_p$
- ▷ Compute the ciphertext: $\mathsf{ct} \coloneqq ([r]_1, r \cdot e(\mathsf{ek}, [1]_2) + m)$[a]
- ▷ Compute the SE-NIZK proof for $\mathcal{R}^{\mathsf{DLOG}}$: $\pi \leftarrow \mathsf{NIZK}^{\mathsf{DLOG}}.\mathsf{Prove}((\mathsf{ct}[1], \mathsf{ct}[2]), r)$
- ▷ Output (ct, π)

$\mathsf{PreDec}\big(\mathsf{sk}, (\mathsf{ct}_i, \pi_i)_{i\in[\ell]}\big)$: (Same as in Construction 1 with different SE-NIZK)

- ▷ Check the SE-NIZK proofs: $\mathsf{NIZK}^{\mathsf{DLOG}}.\mathsf{Verify}((\mathsf{ct}_i[1], \mathsf{ct}_i[2]), \pi_i) \stackrel{?}{=} 1$ for all $i \in [\ell]$
- ▷ Output $\perp$ if any proof is invalid
- ▷ Else output $\mathsf{sbk} \coloneqq \sum_{i\in[\ell]} p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1]$

$\mathsf{Dec}\big(\mathsf{dk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}\big)$:

parse $\mathsf{dk} =: \Big(\big([p_i(x)]_2\big)_{i\in[\ell]}, \big([w_i(x)]_2\big)_{i\in[\ell]}, [p_{-1}(x)+p_0(x)]_2\Big)$

- ▷ For each $j \in [\ell]$, compute, $\mathsf{tm}_j \coloneqq e\big(\mathsf{ct}_j[1],\ [p_j(x)]_2\big)$
- ▷ For each $j \in [\ell]$,
  - Compute

$$L_{j,1} \coloneqq e(\mathsf{ct}_j[1], [w_j(x)]_2) - e\left(\mathsf{sbk} + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{ct}_i[1], [p_j(x)]_2\right) + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{tm}_i$$

$$L_{j,2} \coloneqq e\big(\mathsf{ct}_j[1], [p_{-1}(x)+p_0(x)]_2\big)$$

  - Solve for $r_j \cdot [p_0(x)]_T$ using $L_{j,1}$ and $L_{j,2}$ as $R \coloneqq j((j+1)L_{j,1} - L_{j,2})$.
  - Set $m_j \coloneqq \mathsf{ct}_j[2] - R$
- ▷ Output $(m_j)_{j\in[\ell]}$

[a] One can avoid the pairing during encryption by simply publishing $[p_0(x)]_T$ instead of $[p_0(x)]_1$ as ek.

Construction 5: $\mathsf{BE}_{\mathrm{short}}$: Batch Encryption Construction With Shorter Ciphertexts

$\mathsf{Setup}(1^\lambda)$:

▷ Output $\perp$

$\mathsf{Prove}(x, w)$:

▷ Parse $x =: (h, y)$

▷ Sample $w' \xleftarrow{\$} \mathcal{W}$ and let $x' := [w']_1$.

▷ Let $c \leftarrow H(h, y, x')$

▷ Let $\hat{w} := w' + cw$

▷ Output $\pi := (\hat{w}, x')$

$\mathsf{Verify}(x, \pi)$:

▷ Parse $x =: (h, y)$ and $\pi =: (\hat{w}, x')$

▷ Compute $c \leftarrow H(h, y, x')$

▷ Output $(x' + c \cdot h \stackrel{?}{=} [\hat{w}]_1)$.

Construction 6: NIZK: SE-NIZK construction for Knowledge of Discrete Log

5.1 Correctness

By the completeness of $\mathsf{NIZK}^{\mathsf{DLOG}}$, $\mathsf{NIZK}^{\mathsf{DLOG}}.\mathsf{Verify}(\mathsf{ct}_j, \pi_j) = 1$ for each $\mathsf{ct}_j \leftarrow \mathsf{Enc}(\mathsf{ek}, \mathsf{m}_j)$, $j \in [\ell]$. Let r_j, $j \in [\ell]$, denote the randomness sampled for the ciphertext ct_j during encryption.

Expanding $L_{j,1} = e(\mathsf{ct}_j[1], [w_j(x)]_2) - e\left(\mathsf{sbk} + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{ct}_i[1], [p_j(x)]_2\right) + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{tm}_i$ and using Lemma 1,

$$
\begin{aligned}
&= \left[r_j\left(q_j(x) + \frac{1}{j+1}p_{-1}(x) + \frac{1}{j}p_0(x)\right)\right]_T - \left[\left(\sum_{i\in[\ell]} r_i p_i(x) + \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{r_i}{j-i}\right)p_j(x)\right]_T \\
&\quad + \left[\sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{r_i}{j-i}p_i(x)\right]_T \\
&= [r_j q_j(x)]_T + \left[\frac{r_j}{j+1}p_{-1}(x) + \frac{r_j}{j}p_0(x)\right]_T - \left[r_j q_j(x) + \sum_{\substack{i\in[\ell]\\ i\neq j}}\left(r_i p_i(x) + \frac{r_i}{j-i}\right)p_j(x)\right]_T \\
&\quad + \left[\sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{r_i}{j-i}p_i(x)\right]_T \\
&= \frac{1}{j+1}[r_j p_{-1}(x)]_T + \frac{1}{j}[r_j p_0(x))]_T \ .
\end{aligned}
$$

And, $L_{j,2} = e(ct_j[1], [p_{-1}(x) + p_0(x)]_2) = [r_j p_{-1}(x)]_T + [r_j p_0(x)]_T$.

Therefore, $L_{j,1}$ and $L_{j,2}$ evaluate to the same group elements as in Section 4.1.1. Correctness follows.

5.2 Decryption Complexity

$L_{j,1}$ requires only two pairings to compute, while $L_{j,2}$ requires only one. Thus, the (amortized) number of pairings per ciphertext becomes 4. This can be reduced to 3 pairings by batching $e\big(\mathsf{ct}_j[1], [p_{-1}(x) + p_0(x)]_2\big)$ and $e(\mathsf{ct}_j[1], [w_j(x)]_2)$.

As shown in Section 4.1.2, the MSMs

$$\sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{ct}_i[1] \qquad \sum_{\substack{i\in[\ell]\\ i\neq j}} \frac{1}{j-i}\mathsf{tm}_i$$

can be computed for all the ℓ ciphertexts with only $O(\ell \log \ell)$ $\mathbb{G}_1$ and $\mathbb{G}_T$ operations respectively by using roots of unity for the indices and FFTs to compute the MSMs.

Although Construction 5 is written for a batch of size exactly ℓ, the same algorithm applies to any active batch of size $B \leq \ell$ by replacing all sums over $[\ell]$ with sums over the actual batch. With a nested FFT-friendly choice of index domains, the two Cauchy sums in decryption are computed over the active domain, giving $O(B \log B)$ group operations rather than $O(\ell \log \ell)$.

5.3 Security

To prove security, we introduce a variant of q-StatRankDef (Assumption 1) to account for the modified public parameters and distributions. Informally, the assumption says that it is hard to solve for a designated *target* linear fraction given linear relations over linear fractions but masked using quadratic fractions. Intuitively, this seems natural due to the property of rational functions. Formally, we prove its generic hardness in Appendix A.2.

Assumption 2 (Quadratic Decisional Bilinear Rank-Deficient Diffie-Hellman)**.** Let $x \xleftarrow{\$} \mathbb{Z}_p$ and $q \geq 1$, and let $\mathcal{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$ be a bilinear group. Define,

$$\mathcal{D}_R := \left\{ \left([r]_1,\ [u]_T \right) \;\middle|\; r, u \xleftarrow{\$} \mathbb{Z}_p \right\}, \text{ and}$$
$$\mathcal{D}_t := \left\{ \left([r]_1,\ r \cdot \left[\frac{1}{x}\right]_T \right) \;\middle|\; r \xleftarrow{\$} \mathbb{Z}_p \right\}$$

We state the q-Quadratic-Decisional-Bilinear-Rank-Deficient-Diffie-Hellman (or q-QuadRankDef for short) problem as follows: given

$$\mathsf{pp} := \Bigg(\left(\left[\frac{1}{x+i}\right]_1 \right)_{i=-1}^{q}, \left(\left[\frac{1}{x+i}\right]_2 \right)_{i=1}^{q}, \\ \left(\left[\frac{1}{(x+i)^2} + \frac{1}{i+1}\cdot\frac{1}{x-1} + \frac{1}{i}\cdot\frac{1}{x}\right]_2 \right)_{i=1}^{q}, \left[\frac{1}{x-1} + \frac{1}{x}\right]_2 \Bigg),$$

and a sample ch, decide if $\mathsf{ch} \in \mathcal{D}_R$ or $\mathsf{ch} \in \mathcal{D}_t$. Let,

$$\mathsf{Adv}^{\mathsf{QuadRankDef}}_{q,\mathcal{A}} := \left| \Pr\left[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_R \right] - \Pr\left[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t \right] \right|$$

We say that the q-QuadRankDef assumption holds in $\mathcal{G}$ if for every PPT algorithm $\mathcal{A}$, and for every polynomially bounded $q\colon \mathbb{Z} \to \mathbb{Z}$,

$$\mathsf{Adv}^{\mathsf{QuadRankDef}}_{q(\lambda),\mathcal{A}(\lambda)} = \mathsf{negl}(\lambda)$$

We show that Construction 5 is B-IND-CCA-secure (Definition 3) using the zero-knowledge and straightline simulation-extractability properties of $\mathsf{NIZK}^{\mathsf{DLOG}}$ and the hardness of q-QuadRankDef.

Theorem 4. *The BE construction presented in Construction 5 is* B-IND-CCA*-secure assuming that q-QuadRankDef holds and* $\mathsf{NIZK}^{\mathsf{DLOG}}$ *is complete, zero-knowledge and simulation-extractable.*

The proof is nearly identical to the proof of security of BE in Section 4.1.3. We provide a full proof in Appendix A.3.

6 Experimental Evaluation

We implemented Constructions 1 and 5 in Rust using the `arkworks` library [6] for elliptic-curve arithmetic, pairings, and field operations. The code is publicly available on GitHub [11]. All library code is generic over any compatible pairing curve; benchmarks and concrete numbers below use BLS12-381, and are run on a 2024 MacBook Pro with an Apple M4 Pro processor (12 cores) and 24 GB of unified memory in single-threaded mode.

The code is prototypical—it is not significantly optimized—but already shows competitive numbers for real-world use; we note one optimization: per ciphertext, the mask R computed from $L_{j,1}$ and $L_{j,2}$ share one final exponentiation across two Miller loops. This saves ℓ final exponentiations per batch.

Table 2 reports wall-clock timings for encryption, pre-decryption and decryption for both Constructions 1 and 5, as a function of the batch size ℓ. Encryption is independent of the batch size and is slightly faster for Construction 5 due to one fewer $\mathbb{G}_1$ scalar multiplication. Pre-decryption times are the same for both constructions and involve 1 $\mathbb{G}_1$ MSM of size ℓ. Decryption is roughly 1.6 times faster in Construction 5 than Construction 1 due to one fewer $\mathbb{G}_2$ FFT and ℓ fewer pairings total across the batch.

	Enc		PreDec		Dec	
ℓ	BE	$\mathsf{BE}_{\mathrm{short}}$	BE	$\mathsf{BE}_{\mathrm{short}}$	BE	$\mathsf{BE}_{\mathrm{short}}$
4	599 µs	437 µs	1.53 ms	0.82 ms	19.2 ms	15.3 ms
16	599 µs	437 µs	2.83 ms	1.93 ms	101 ms	76.3 ms
64	599 µs	437 µs	8.12 ms	6.93 ms	501 ms	364 ms
128	599 µs	437 µs	14.5 ms	13.2 ms	1.09 s	787 ms
256	599 µs	437 µs	27.9 ms	25.6 ms	2.38 s	1.69 s
512	599 µs	437 µs	54.3 ms	50.1 ms	5.12 s	3.61 s

Table 2: Mean running times for encryption, pre-decryption, and decryption in BE (Construction 1) and $\mathsf{BE}_{\mathrm{short}}$ (Construction 5).

7 Acknowledgments

This work was partially supported by a grant from UBRI.

References

1. Michel Abdalla, Fabrice Benhamouda, and Philip MacKenzie. Security of the j-pake password-authenticated key exchange protocol. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*, SP '15, page 571–587, USA, 2015. IEEE Computer Society.
2. Amit Agarwal, Kushal Babel, Sourav Das, and Babak Poorebrahim Gilkalaye. Time-lock encrypted storage for blockchains. Cryptology ePrint Archive, Paper 2025/2048, 2025.
3. Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Paper 2025/2115, 2025.
4. Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Paper 2026/754, 2026.
5. Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In *CRYPTO (3)*, volume 16002 of *Lecture Notes in Computer Science*, pages 69–100. Springer, 2025.
6. arkworks contributors. `arkworks` zksnark ecosystem. https://arkworks.rs, 2022.
7. Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022.
8. Elwyn R. Berlekamp. Factoring polynomials over finite fields. *Bell System Technical Journal*, 46(8):1853–1859, 1967.
9. Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer, 2004.
10. Dan Boneh, Evan Laufer, and Ertem Nusret Tas. Batch decryption without epochs and its application to encrypted mempools. Cryptology ePrint Archive, Paper 2025/1254, 2025.
11. Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Implementation of Efficient Batch Threshold Encryption Using Partial Fraction Techniques, 2026. https://github.com/entrohpy/batch-enc-partial-fractions.
12. Dan Boneh and Victor Shoup. A graduate course in applied cryptography. https://toc.cryptobook.us/book.pdf, 2023.
13. Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.
14. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: epochless approach to batched threshold encryption for MEV prevention. In *USENIX Security Symposium*, pages 3457–3476. USENIX Association, 2025.
15. Jan Camenisch, Maria Dubovitskaya, Kristiyan Haralambiev, and Markulf Kohlweiss. Composable and modular anonymous credentials: Definitions and practical constructions. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology – ASIACRYPT 2015*, pages 262–288, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
16. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *USENIX Security Symposium*. USENIX Association, 2024.
17. Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In *USENIX Security Symposium*, pages 3477–3495. USENIX Association, 2025.
18. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *SP*, pages 910–927. IEEE, 2020.
19. Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In *EUROCRYPT (2)*, Lecture Notes in Computer Science, pages 311–341. Springer, 2024.
20. Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. Cryptology ePrint Archive, Paper 2025/2032, 2025.
21. Georg Fuchsbauer, Antoine Plouviez, and Yannick Seurin. Blind schnorr signatures and signed elgamal encryption in the algebraic group model. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 63–95, Cham, 2020. Springer International Publishing.
22. Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. Cryptology ePrint Archive, Paper 2025/2103, 2025.
23. Feng Hao. Schnorr non-interactive zero-knowledge proof. RFC 8235, 2017.

24. Charanjit S. Jutla, Rohit Nema, and Arnab Roy. Partial fraction techniques for cryptography. Cryptology ePrint Archive, Paper 2025/2081, 2025.
25. Guru-Vamsi Policharla. A simple batched threshold encryption scheme. Cryptology ePrint Archive, Paper 2026/760, 2026.
26. Michael K. Reiter and Kenneth P. Birman. How to securely replicate services. *ACM Trans. Program. Lang. Syst.*, 16(3):986–1009, 1994.
27. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997.
28. Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. Cambridge University Press, 2 edition, 2009.
29. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In Kaisa Nyberg, editor, *Advances in Cryptology — EUROCRYPT'98*, pages 1–16, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
30. Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762, 2024.
31. Hong Wang, Yuqing Zhang, and Dengguo Feng. Short threshold signature schemes without random oracles. In *INDOCRYPT 2005*, volume 3797 of *Lecture Notes in Computer Science*, pages 297–310. Springer, 2005.
32. Saisi Xiong, Yijian Zhang, and Jie Chen. Efficient batched IBE from lattices in the standard model. Cryptology ePrint Archive, Paper 2025/2158, 2025.

A Additional Proofs

A.1 Hardness of q-StatRankDef (Assumption 1)

We reduce q-$\mathsf{StatRankDef}$ to the q-Bilinear-Decisional-Rank-Deficient-Diffie-Hellman problem (or q-$\mathsf{RankDef}$ for short) [24, Assumption 3], which was shown to be secure in the GBGM. Formally,

Lemma 6. *q-StatRankDef holds if q-$\mathsf{RankDef}$ holds.*

Proof. We proceed with a proof by contradiction. Suppose q-$\mathsf{StatRankDef}$ does not hold. That is, there exists a PPT adversary $\mathcal{A}$ such that $\mathsf{Adv}^{\mathsf{StatRankDef}}_{q,\mathcal{A}} > \mathsf{negl}(\lambda)$. We construct an adversary $\mathcal{B}$ using $\mathcal{A}$ such that $\mathsf{Adv}^{\mathsf{RankDef}}_{q+2,\mathcal{B}} = \frac{1}{2}\mathsf{Adv}^{\mathsf{StatRankDef}}_{q,\mathcal{A}}$.

Let $\mathsf{Ch}^{\mathsf{RankDef}}$ be the challenger for the q-$\mathsf{RankDef}$ problem. $\mathcal{B}$ proceeds as follows.

Send $\mathsf{Ch}^{\mathsf{RankDef}}$ the following queries:

- $(\mathsf{target}, 0)$ and receive $[p_0(x)]_1$ for a $x \xleftarrow{\$} \mathbb{Z}_p$ sampled by $\mathsf{Ch}^{\mathsf{RankDef}}$;
- $(1, i)$ and $(2, i)$ for $i \in [q]$ to receive $[p_i(x)]_1$ and $[p_i(x)]_2$ respectively;
- $(1, -1)$ to receive $[p_{-1}(x)]_1$.

Finally, $\mathcal{B}$ sends the query $(\mathsf{Combine}, \{-1\} \cup [q])$ to receive challenge $(\mathsf{ch}_1, \mathsf{ch}_2, \mathsf{ch}_3)$. Let $\mathsf{pp} := \big([p_{-1}(x)]_1, [p_0(x)]_1, [p_1(x)]_1, \ldots, [p_q(x)]_1, [p_1(x)]_2, \ldots, [p_q(x)]_2\big)$, and $\mathsf{ch} := (\mathsf{ch}_1, \mathsf{ch}_2, \mathsf{ch}_3)$.

Send pp and ch to $\mathcal{A}$. If $\mathcal{A}$ outputs 0 (*i.e.*, sample is from $\mathcal{D}_R$), $\mathcal{B}$ outputs $b' = 1$. Else if $\mathcal{A}$ outputs 1 (*i.e.*, sample is from $\mathcal{D}_t$), $\mathcal{B}$ outputs $b' = 0$. We claim $\mathsf{Adv}^{\mathsf{RankDef}}_{q+2,\mathcal{B}} = \frac{1}{2}\mathsf{Adv}^{\mathsf{StatRankDef}}_{q,\mathcal{A}}$. To see this, first observe that pp above is identically distributed to pp in the q-$\mathsf{StatRankDef}$ problem in Assumption 1. Second, $\mathsf{Ch}^{\mathsf{RankDef}}$ generates $\mathsf{ch}_1 = [r]_1$, $\mathsf{ch}_2 = r \cdot \sum_{i=-1}^{q} [p_i(x)]_1$ for $r \xleftarrow{\$} \mathbb{Z}_p$ by the $(q+2)$-$\mathsf{RankDef}$ problem formulation. If $\mathsf{Ch}^{\mathsf{RankDef}}$ samples internal bit $b = 0$, then $\mathsf{ch}_3 = r[p_0(x)]_T$ and ch is identically distributed to a challenge sample from $\mathcal{D}_t$; else $\mathsf{ch}_3 = [r']_T$ for $r' \xleftarrow{\$} \mathbb{Z}_p$ and ch is identically distributed to a challenge sample from $\mathcal{D}_R$.

Therefore, we obtain,

$$
\begin{aligned}
\mathsf{Adv}^{\mathsf{RankDef}}_{q+2,\mathcal{B}} &= \left| \Pr\big[b' = b\big] - \frac{1}{2} \right| \\
&= \left| \frac{1}{2}\Pr\big[b' = 0 \mid b = 0\big] + \frac{1}{2}\Pr\big[b' = 1 \mid b = 1\big] - \frac{1}{2} \right| \\
&= \left| \frac{1}{2}\Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \mid \mathsf{ch} \leftarrow \mathcal{D}_t] + \frac{1}{2}\Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 0 \mid \mathsf{ch} \leftarrow \mathcal{D}_R] - \frac{1}{2} \right| \\
&= \left| \frac{1}{2}\Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \mid \mathsf{ch} \leftarrow \mathcal{D}_t] + \frac{1}{2}(1 - \Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \mid \mathsf{ch} \leftarrow \mathcal{D}_R]) - \frac{1}{2} \right| \\
&= \frac{1}{2}\left|\Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \mid \mathsf{ch} \leftarrow \mathcal{D}_t] - \Pr[\mathcal{A}(\mathsf{pp}, \mathsf{ch}) = 1 \mid \mathsf{ch} \leftarrow \mathcal{D}_R]\right| \\
&= \frac{1}{2}\mathsf{Adv}^{\mathsf{StatRankDef}}_{q,\mathcal{A}}
\end{aligned}
$$

Therefore, $\mathsf{Adv}^{\mathsf{RankDef}}_{q+2,\mathcal{B}} > \mathsf{negl}(\lambda)$, a contradiction. □

A.2 Generic Hardness of q-QuadRankDef (Assumption 2)

Lemma 7. *Assumption 2 holds in the Generic Bilinear Group Model (GBGM).*

Proof sketch. In the GBGM, every $\mathbb{G}_T$ element computable by $\mathcal{A}$ is a bilinear combination

$$\sum_{\alpha,\beta} c_{\alpha\beta}\, f_\alpha(x)\, h_\beta(x),$$

where $\{f_\alpha\}$ ranges over the $\mathbb{G}_1$-basis and $\{h_\beta\}$ over the $\mathbb{G}_2$-basis. The $\mathbb{G}_1$-basis is $\{p_i(X)\}_{i=-1}^{q}$ and the $\mathbb{G}_2$-basis is $\{1, p_1(X), \ldots, p_q(X), g(X), w_1(X), \ldots, w_q(X)\}$.

A computable $\mathbb{G}_T$ element involving the challenge randomness r must arise from pairing $[r]_1$ with some $\mathbb{G}_2$-basis element. We claim $p_0(X) \notin \mathrm{span}_{\mathbb{F}}\{1, p_1, \ldots, p_q, g, w_1, \ldots, w_q\}$. Suppose otherwise:

$$p_0(X) = \alpha_0 + \sum_{j=1}^{q} \alpha_j\, p_j(X) + \beta\, g(X) + \sum_{j=1}^{q} \gamma_j\, w_j(X).$$

Each w_j has a double pole at $X = -j$, while $p_0(X) = 1/X$ has no double poles. Since no other term on the right can cancel a double pole at $X = -j$, we must have $\gamma_j = 0$ for all j. Then $g(X) = p_{-1}(X) + p_0(X)$ has a simple pole at $X = 1$ (from $p_{-1}(X) = 1/(X-1)$), while no p_j for $j \geq 1$ has such a pole, so $\beta = 0$. The remaining terms $\alpha_0 + \sum_j \alpha_j p_j(X)$ have poles only at $\{-1, \ldots, -q\}$, never at $X = 0$, so they cannot equal $p_0(X)$—a contradiction.

Hence no pairing involving $[r]_1$ yields $r \cdot p_0(x)$, and in the GBGM, the challenge $r \cdot [p_0(x)]_T$ is information-theoretically hidden. □

A.3 CCA-2 Security of $\mathsf{BE}_{\mathbf{short}}$

We prove Theorem 4 below.

Proof. Suppose for contradiction $\mathsf{BE}_{\mathrm{short}}$ is not $\mathsf{B\text{-}IND\text{-}CCA}$-secure. Then, there exists $\ell \in \mathbb{N}$ and PPT adversary $\mathcal{A}$ that wins the game for batch encryption in Figure 2 with non-negligible probability, *i.e.*, $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A},\ell} > \mathsf{negl}(\lambda)$. We construct PPT adversary $\mathcal{B}$ such that $\mathsf{Adv}^{\mathsf{QuadRankDef}}_{\ell,\mathcal{B}} > \mathsf{negl}(\lambda)$.

$\mathcal{B}$ receives as input $3\ell + 3$ group elements of the form

$$([p_i(x)]_1)_{i=-1}^{\ell}, ([p_i(x)]_2)_{i\in[\ell]}, ([w_i(x)]_2)_{i\in[\ell]}, [g(x)]_2,$$

where $p_i(X) = \frac{1}{X+i}$, $w_i(X) = (p_i(X))^2 + \frac{1}{i+1}p_{-1}(X) + \frac{1}{i}p_0(X)$, $g(X) = p_{-1}(X) + p_0(X)$, and a challenge $\mathsf{ch} = (\mathsf{ch}_1, \mathsf{ch}_2)$ from the ℓ-$\mathsf{QuadRankDef}$ challenger. It sends $\mathsf{ek} := [p_0(x)]_1$ and $\mathsf{dk} := \Big(([p_i(x)]_2)_{i\in[\ell]}, ([w_i(x)]_2)_{i\in[\ell]}, [g(x)]_2\Big)$ to $\mathcal{A}$. $\mathcal{B}$ also runs $(\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{td}_{\mathsf{Ext}}) \leftarrow \mathsf{NIZK}^{\mathsf{DLOG}}.\mathsf{SimSetup}(1^\lambda)$.

$\mathcal{B}$ responds to $\mathcal{A}$'s subsequent queries as follows,

- Pre-challenge queries: $\mathcal{A}$ sends a list $(\mathsf{ct}_i, \pi_i)_{i\in[\ell]}$. If $\mathsf{NIZK}^{\mathsf{DLOG}}.\mathsf{Verify}(\mathsf{ct}_i, \pi_i) = 0$ for some $i \in [\ell]$, $\mathcal{B}$ responds with $\perp$, as PreDec would. Otherwise, $\mathcal{B}$ runs the (straightline) extractor Ext from the simulation extractability of $\mathsf{NIZK}^{\mathsf{DLOG}}$ (Definition 13) for each $i \in [\ell]$: $r_i \leftarrow \mathsf{Ext}(\mathsf{crs}, \mathsf{td}_{\mathsf{Ext}}, \mathsf{ct}_i, \pi_i)$, where the statement is the entire ciphertext $\mathsf{ct}_i = (\mathsf{ct}_i[1], \mathsf{ct}_i[2])$. It responds to $\mathcal{A}$ with $\mathsf{sbk} := \sum_{i\in[\ell]} r_i \cdot [p_i(x)]_1$.

– Challenge round: $\mathcal{A}$ sends $(\mathsf{m}_0, \mathsf{m}_1)$. $\mathcal{B}$ samples bit $b \xleftarrow{\$} \{0,1\}$ and computes the ciphertext using the ℓ-QuadRankDef challenge: $\mathsf{ct}^* := (\mathsf{ch}_1, \mathsf{ch}_2 + \mathsf{m}_b)$. $\mathcal{B}$ generates a simulated proof on the statement ct^* using the simulator of the NIZK (Definitions 12 and 13): $\pi^* \leftarrow \mathsf{SimProve}(\mathsf{crs}, \mathsf{td}_{\mathsf{Sim}}, \mathsf{ct}^*)$, and returns the challenge ciphertext (ct^*, π^*) to $\mathcal{A}$.
– Post-challenge queries: $\mathcal{A}$ sends a list $(\mathsf{ct}_i, \pi_i)_{i\in[\ell]}$. If the list contains the challenge ciphertext (ct^*, π^*), $\mathcal{B}$ responds with $\perp$, as the challenger in Figure 2 would. Otherwise, $\mathcal{B}$ responds as in the pre-challenge queries.

$\mathcal{A}$ outputs a bit b'. If $b = b'$, $\mathcal{B}$ outputs 1. Else it outputs 0 or 1 with equal probability.

Extraction step. Every pair (ct_i, π_i) on which $\mathcal{B}$ runs Ext lies outside the set Q of statement–proof pairs output by $\mathsf{SimProve}$. Indeed, in the pre-challenge phase, $Q = \emptyset$, and in the post-challenge phase, $Q = \{(\mathsf{ct}^*, \pi^*)\}$ and all ciphertexts in batches that receive $\mathsf{sbk} \neq \perp$ differ from the challenge ciphertext. Hence, if with non-negligible probability ϵ some verifying queried pair yielded an invalid witness $(\mathsf{ct}_i[1] \neq [r_i]_1)$, an adversary $\mathcal{A}_{\mathsf{SE}}$ that simulates $\mathcal{B}$'s game (sampling x and the ℓ-QuadRankDef challenge itself, and obtaining π^* from its simulation oracle) and outputs a uniformly guessed queried pair would win the game of Definition 13 with non-negligible probability of at least $\epsilon/(\ell\, q(\lambda))$, where $q(\lambda)$ bounds the number of $\mathcal{A}$'s queries. This contradicts the simulation-extractability of $\mathsf{NIZK}^{\mathsf{DLOG}}$. Therefore, except with negligible probability, all extracted witnesses are valid, $\mathsf{ct}_i[1] = [r_i]_1$ implies $p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1] = r_i \cdot [p_i(x)]_1$ for every $i \in [\ell]$, and thus $\mathsf{sbk} = \sum_{i\in[\ell]} r_i \cdot [p_i(x)]_1 = \sum_{i\in[\ell]} p_i(\mathsf{sk}) \cdot \mathsf{ct}_i[1]$, as claimed.

We next prove the claim $\mathsf{Adv}^{\mathsf{QuadRankDef}}_{\ell,\mathcal{B}} \geq \frac{1}{2}\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A},\ell} - \mathsf{negl}(\lambda)$ by considering the following two cases:

ℓ-QuadRankDef challenger samples from $\mathcal{D}_t$: By the zero-knowledge property of $\mathsf{NIZK}^{\mathsf{DLOG}}$, the game $\mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A},\ell}$, where the proof π^* is generated by the algorithm Prove and the game, where the same proof is output by Sim, are indistinguishable for $\mathcal{A}$ except with probability $\mathsf{negl}(\lambda)$. Now, observe that in $\mathcal{B}$'s simulation, ek is identically distributed to the output of Setup for $\mathsf{BE}_{\mathrm{short}}$. Moreover, by the extraction step above, $\mathcal{B}$'s responses to the pre- and post-challenge queries are equal to those of PreDec except with probability $\mathsf{negl}(\lambda)$. Finally, since the ℓ-QuadRankDef challenger samples from $\mathcal{D}_t$, the challenge ciphertext ct^*, constructed by $\mathcal{B}$, is identically distributed to that of Enc. Therefore, $\mathcal{A}$'s transcript in $\mathcal{B}$'s simulation is indistinguishable from that of $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}}$ in Figure 2 except with probability $\mathsf{negl}(\lambda)$.

ℓ-QuadRankDef challenger samples from $\mathcal{D}_R$: By the same arguments above, $\mathcal{A}$'s transcript in $\mathcal{B}$'s simulation is indistinguishable from that of $\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}}$ in Figure 2 except with probability $\mathsf{negl}(\lambda)$. Moreover, since the ℓ-QuadRankDef challenger samples from $\mathcal{D}_R$, the ciphertext ct^* perfectly hides m_b as it is independent of b. Thus, $\mathcal{A}$ outputs $b' = b$ with probability $\frac{1}{2}$; since b is chosen uniformly at random. Then, the probability that $\mathcal{B}$ outputs 1 is, $\Pr[b = b'] \cdot \Pr[\mathcal{B} \text{ outputs } 1 | b = b'] + \Pr[b \neq b'] \cdot \Pr[\mathcal{B} \text{ outputs } 1 | b \neq b'] = \frac{3}{4}$.

So $\mathsf{Adv}^{\mathsf{QuadRankDef}}_{\ell,\mathcal{B}}$ is,

$$
\begin{aligned}
&\left|\Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_R\right] - \Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t\right]\right| \\
&\geq \left|\frac{3}{4} - \left(\Pr\left[b = b'\right] \cdot \Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t, b = b'\right] + \right.\right. \\
&\qquad \left.\left. \Pr\left[b \neq b'\right] \cdot \Pr\left[\mathcal{B}(\mathsf{pp},\mathsf{ch}) = 1 \;\middle|\; \mathsf{ch} \xleftarrow{\$} \mathcal{D}_t, b \neq b'\right]\right)\right| - \mathsf{negl}(\lambda)
\end{aligned}
$$

$$
\begin{aligned}
&\geq \left| \frac{3}{4} - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}}(1^\lambda, 1^\ell) = 1] \cdot 1 \right. \\
&\qquad \left. - (1 - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}}(1^\lambda, 1^\ell) = 1]) \cdot \frac{1}{2} \right| - \mathsf{negl}(\lambda) \\
&= \frac{1}{2} \left| \frac{1}{2} - \Pr[\mathsf{Game}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}}(1^\lambda, 1^\ell) = 1] \right| - \mathsf{negl}(\lambda) \\
&= \frac{1}{2} \mathsf{Adv}^{\mathsf{B\text{-}IND\text{-}CCA}}_{\mathsf{BE}_{\mathrm{short}},\mathcal{A}} - \mathsf{negl}(\lambda)
\end{aligned}
$$

Therefore, $\mathsf{Adv}^{\mathsf{QuadRankDef}}_{\ell,\mathcal{B}} > \mathsf{negl}(\lambda)$, a contradiction. □